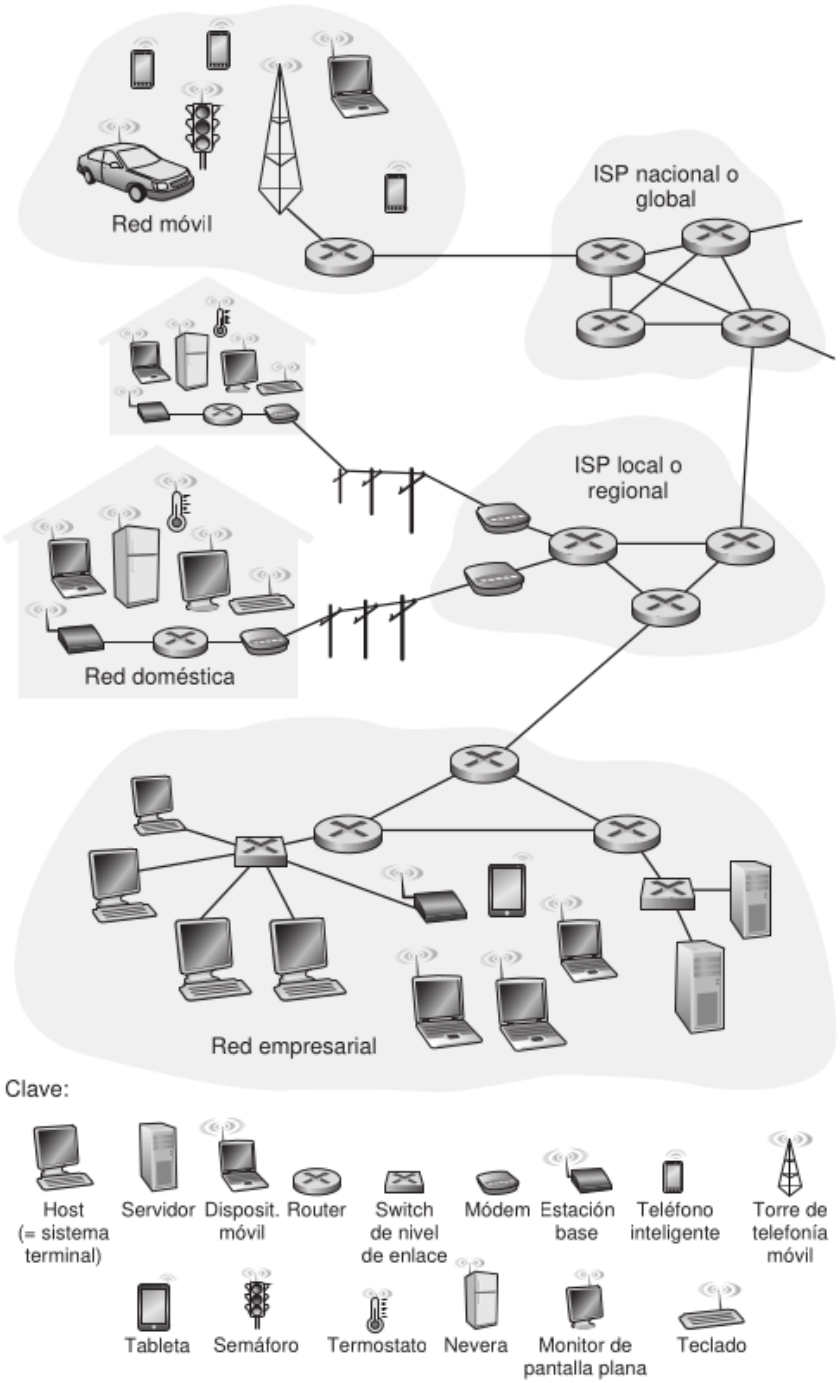
Resumen / Conceptos tomados a criterio del libro:
“computer-networking-a-top-down-approach-8th-edition”

Nombre: Joaquin Embon
Materia: Redes
Cátedra: Hamelin
Cursado: 2c2025

Hasta el capítulo 3 inclusive hago un correcto resumen sobre el libro, a partir del capítulo 4 fui definiendo conceptos con el libro y apuntes tomados en clase.
(Recomiendo leer el libro).

Capítulo 1 - Computer Networks and The Internet

host o end systems: dispositivos conectados a la red (laptop, tablet, reloj, etc).
Estos últimos conectados por una red de communication links y packet switches.



Los **links** transmiten de acuerdo a una transmission rate medida en bits/segundo.
Packet: paquete de información a ser enviado a través de la red.

El **packet switch** toma el packet que viene de su communication link de entrada y lo redirecciona a uno de salida. Un packet switch *puede ser un* router (en network core) o un link-layer switch(en access network).

El camino recorrido por el packet es el **route** o **path**.

Analogía

- **packets** = camiones.
- **communication links** = rutas.
- **packet switches** = intersecciones.

end systems acceden a internet a través de Internet Service Providers (ISPs). Cada ISP es en sí mismo una red aparte. Los ISPs o lower tier ISPs están interconectados por upper tier ISP (fibra óptica).

Las piezas que componen internet corren protocolos para controlar el envío y recepción de datos. El Transmission Control Protocol (TCP) e Internet Protocol (IP) son los más importantes.

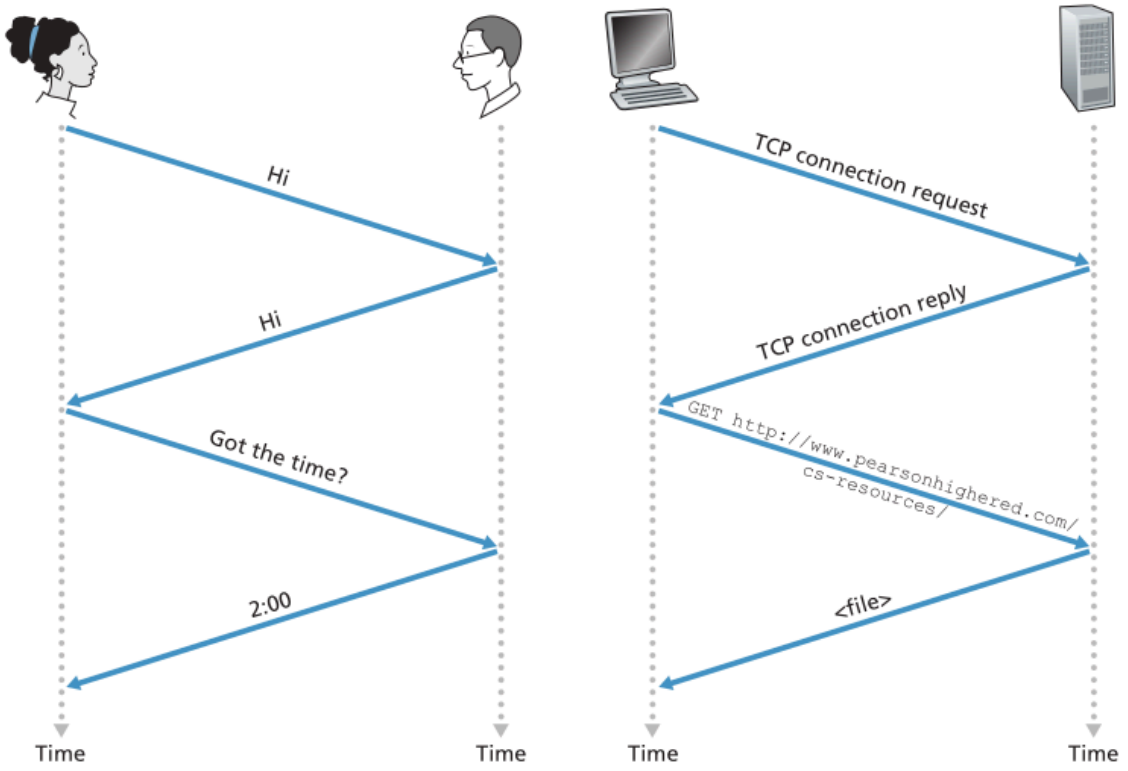
internet standards son desarrollados por la IETF. Estos documentos son llamados **request for comments** (**RFCs** y definen los protocolos). Los RFC nacieron como solicitudes de comentarios de carácter general (de ahí su nombre) para solucionar los problemas de diseño de la red y de los protocolos a los que se enfrentó el precursor de Internet [Allman 2011]. El contenido de estos documentos suele ser bastante técnico y detallado. Definen protocolos tales como TCP, IP, HTTP (para la Web) y SMTP (para el correo electrónico). Actualmente, existen más de 7.000 documentos RFC. Existen también otros organismos dedicados a especificar estándares para componentes de red, especialmente para los enlaces de red.

El **internet** también puede ser descrito como una infraestructura que provee servicios a aplicaciones (**distributed applications**). Corren en end systems. Los **end systems proveen interfaces para sockets que especifican cómo solicitar y enviar información a otro end system** (esto responde a la pregunta de: cómo hace un programa que se ejecuta en un sistema terminal para ordenar a Internet que entregue datos a otro programa que se ejecuta en otro sistema terminal?)

Protocolo

Probablemente, sea más sencillo comprender el concepto de protocolo de red considerando en primer lugar algunas analogías humanas, ya que las personas utilizamos protocolos casi constantemente. Piense en lo que hace cuando necesita preguntar a alguien qué hora es. En la Figura se muestra cómo se lleva a cabo un intercambio de este tipo. El protocolo entre personas (o las buenas maneras, al menos) dicta que para iniciar un proceso de comunicación con alguien lo primero es saludar (el primer “Hola” mostrado en la Figura). La respuesta típica a este saludo será también “Hola”. Implícitamente, el saludo cordial de respuesta se toma como una indicación de que se puede continuar con el proceso de comunicación y preguntar la hora. Una respuesta diferente al “Hola” inicial (como por ejemplo, “¡No me moleste!” o “No hablo su idioma”, o cualquier respuesta impublicable), podría indicar una falta de disposición o una incapacidad para comunicarse. En este caso, el protocolo de las relaciones entre personas establece que no debe preguntarse la hora. En ocasiones, no se obtiene ninguna respuesta, en cuyo caso normalmente renunciamos a preguntar a esa persona la hora que es. Observe que, en el

protocolo entre personas, existen mensajes específicos que enviamos y acciones específicas que tomamos como respuesta a los mensajes de contestación recibidos o a otros sucesos (como, por ejemplo, no recibir una respuesta en un periodo de tiempo determinado). Evidentemente, los mensajes transmitidos y recibidos, y las acciones tomadas al enviar o recibir estos mensajes o al producirse otros sucesos, desempeñan un papel principal en el protocolo humano. Si las personas adoptan protocolos diferentes (por ejemplo, si una persona guarda las formas pero la otra no lo hace, o si uno comprende el concepto de tiempo y el otro no), los protocolos no interoperarán y no podrá llevarse a cabo ninguna tarea útil. Esta misma idea también es aplicable a las redes: hacen falta dos (o más) entidades comunicándose y ejecutando el mismo protocolo para poder llevar a cabo una tarea.

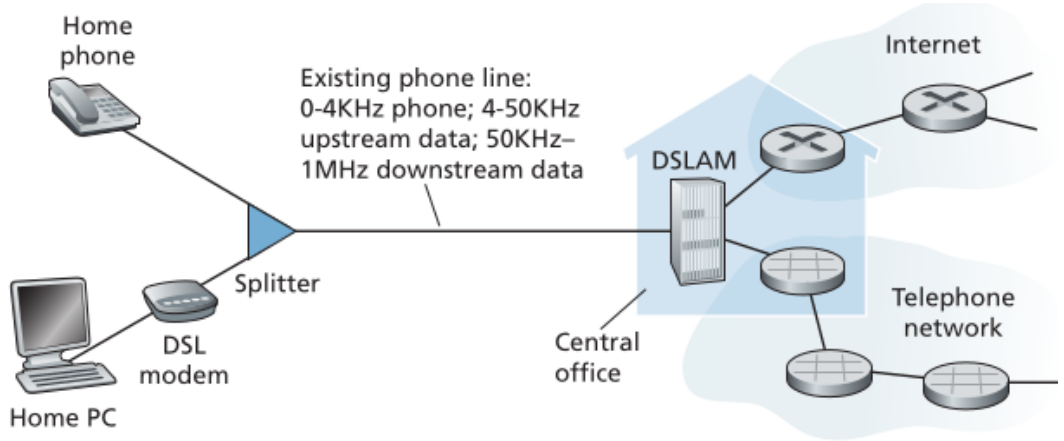


Un **protocolo** define el formato y el orden de los mensajes intercambiados entre dos o más entidades que se comunican, así como las acciones tomadas al producirse la transmisión y/o recepción de un mensaje u otro suceso.

Los **end systems** (sistemas terminales) son también conocidos como hosts, estos se dividen en clientes y servidores. La mayoría de los servidores residen en **data centers.** De un modo informal, podríamos decir que los clientes suelen ser las computadoras de escritorio y portátiles, los teléfonos inteligentes, etc., mientras que los servidores suelen ser equipos más potentes que almacenan y distribuyen páginas web, flujos de vídeo, correo electrónico, etc.

access network: la red que conecta físicamente un host al primer router en su camino a otro end system.

El **acceso** a **internet** residencial más común es por digital subscriber line (**DSL**) (en general lo provee la compañía de teléfono, el módem usa la línea telefónica en este caso) y por cable.



Existen estándares para DSL que definen múltiples **transmission rates** (tasas de transmisión).

Por otra parte, los cables de acceso a internet hacen uso del cable del proveedor de televisión.
Mientras que la tecnología DSL emplea la infraestructura local existente de la compañía telefónica, el acceso por cable a Internet utiliza la infraestructura de televisión por cable existente. Las viviendas obtienen el acceso por cable a Internet de la misma compañía que les proporciona la televisión por cable. Como se ilustra en la Figura siguiente, se usa fibra óptica para conectar el terminal de cabecera del cable a una serie de nodos de área situados en el vecindario, a partir de los cuales se utiliza cable coaxial tradicional para llegar a todos los domicilios. Cada nodo de área suele dar soporte a entre 500 y 5.000 viviendas. Puesto que en este sistema se emplea tanto cable coaxial como fibra, a menudo se denomina sistema HFC (Hybrid Fiber Coax, Híbrido de fibra y coaxial).

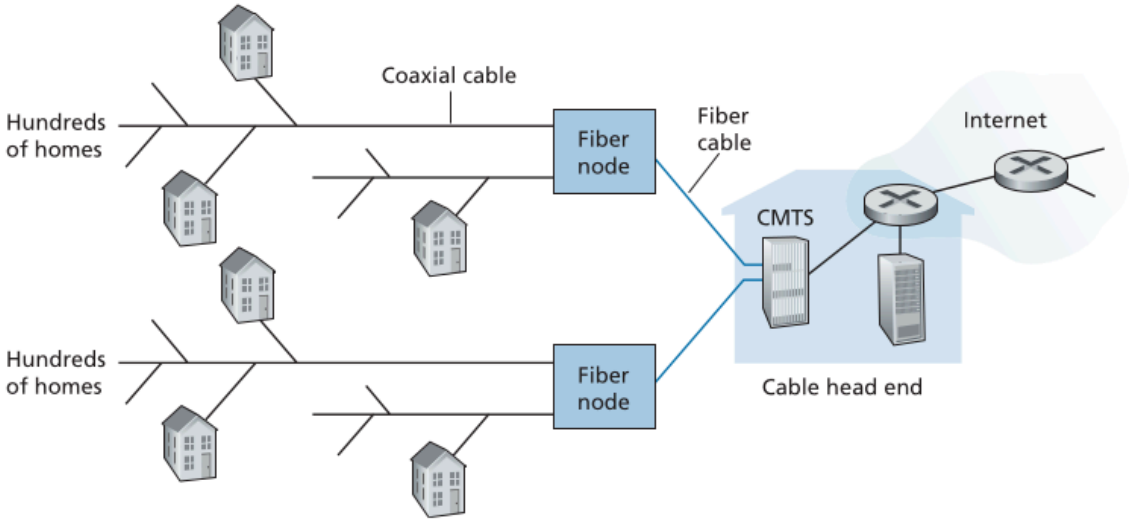


Figura 1.6 ♦ Red de acceso híbrida de fibra óptica y cable coaxial.

requiere módems especiales para cable, en general conectados a la PC por un puerto **ethernet**. En estos casos los packets llegan a todos. Los channel de subida y bajada son compartidos.

Más veloz que los últimos dos pero menos común es fibra óptica en el hogar (**FTTH**)(Fiber-To-The-Home, Fibra hasta el hogar).

Hay disponibles dos arquitecturas de distribución de fibra óptica que llevan a cabo esta separación: las **redes ópticas activas** (AON, Active Optical Network) y las **redes ópticas pasivas** (PON, Passive Optical Network).
La Figura 1.7 muestra un sistema **FTTH** que utiliza la arquitectura de distribución **PON**. Cada vivienda dispone de una terminación de **red óptica** (**ONT**, Optical Network Terminator), que se conecta a un distribuidor del vecindario mediante un **cable de fibra óptica** dedicado. El distribuidor combina una cierta cantidad de viviendas (normalmente menos de 100) en un único cable de fibra óptica compartido, que se conecta a una terminación de línea óptica (**OLT**, Optical Line Terminator) en la central de la compañía telefónica. La OLT, que realiza la conversión de señales ópticas en eléctricas, se conecta a Internet a través de un router de la compañía telefónica. En los domicilios, los usuarios conectan su router doméstico (normalmente un router inalámbrico) con la **ONT** y acceden a Internet a través de este router. En la arquitectura PON, todos los paquetes enviados desde la OLT al distribuidor se replican en este distribuidor (de forma similar a un terminal de cabecera de cable).

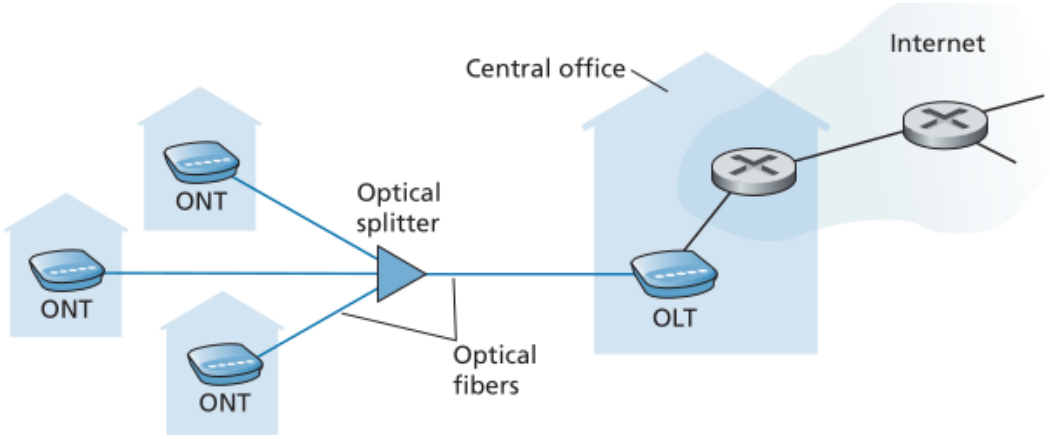


Figure 1.7 ♦ FTTH Internet access

AON (Active Optical Network):
Utiliza **dispositivos electrónicos activos** (como switches o routers) entre el proveedor y los usuarios para distribuir la señal óptica.

PON (Passive Optical Network):
Utiliza **divisores ópticos pasivos (splitters)** que no requieren energía eléctrica para repartir la señal entre varios usuarios.

LAN (local area network) es usada para conectar un end system al primer router (edge). En general se hace vía **ethernet** switch.

Aunque existen muchos tipos de tecnologías LAN, **Ethernet** es con mucho la tecnología de acceso predominante en las redes corporativas, universitarias y domésticas. Como se ilustra en la Figura 1.8, los usuarios de Ethernet utilizan cable de cobre de par trenzado para conectarse a un **switch Ethernet**, el switch Ethernet (o una red de tales switches) se conecta a Internet. Con acceso Ethernet, normalmente los usuarios disponen de velocidades de acceso de 100 Mbps o 1 Gbps al switch Ethernet, mientras que los servidores pueden disponer de acceso de 1 Gbps o incluso 10 Gbps.

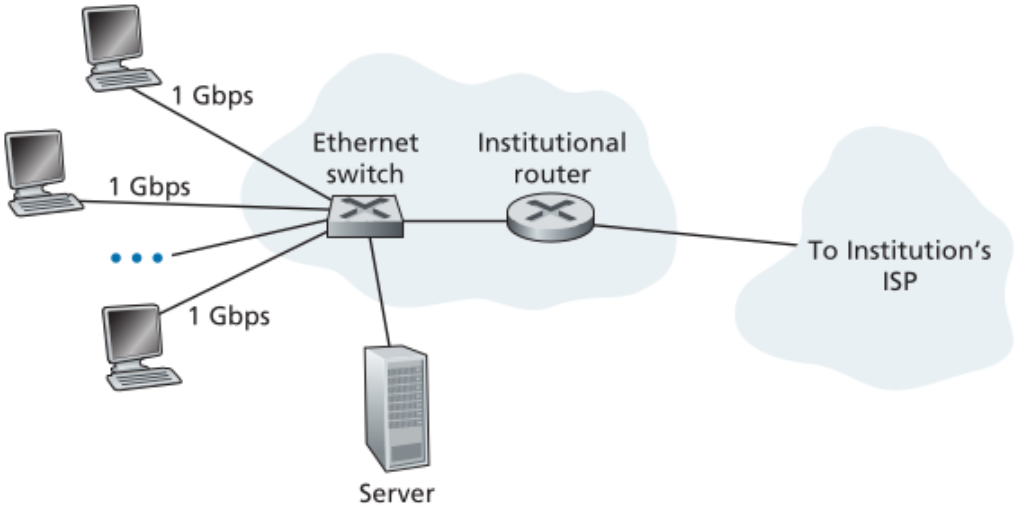


Figure 1.8 ♦ Ethernet Internet access

Muchas viviendas combinan acceso residencial de banda ancha (es decir, módems por cable o DSL) con estas baratas tecnologías de LAN inalámbrica para crear redes domésticas potentes [Edwards 2011]. La Figura 1.9 muestra el esquema de una red doméstica típica. Esta red doméstica está formada por un portátil con función de itinerancia (**roaming**) y un PC de sobremesa; una estación base (el punto de acceso inalámbrico), que se comunica con el portátil y otros dispositivos inalámbricos de la vivienda; un módem por cable, que proporciona el acceso de banda ancha a Internet; y un router, que interconecta la estación base y el PC de sobremesa con el módem por cable. Esta red permite a los miembros de la familia disponer de acceso de banda ancha a Internet, pudiendo uno de ellos acceder mientras se mueve de la cocina a los dormitorios y al jardín.

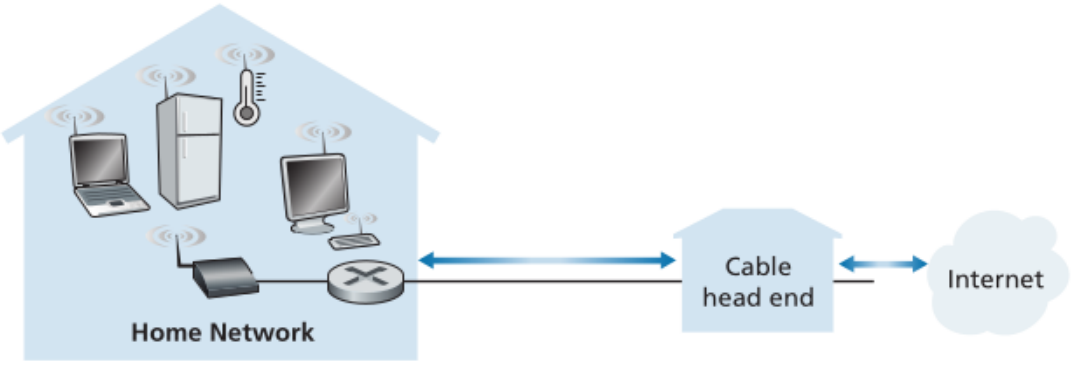


Figure 1.9 ♦ A typical home network

Wide-Area Wireless access: 3g and LTE, mayor alcance que el wifi.

Medios físicos

Los bits se propagan a través de un medio físico: **par trenzado**, **coaxial**, **fibra óptica**, **espectro radial**, etc. Puede ser un **medio guiado** o **no**.

Guided media: Las ondas son guiadas a través de un medio físico (cable).

Unguided media: En el caso de no ser guiado, las ondas se propagan por la atmósfera y espacio (wireless LAN).

Cable de cobre de par trenzado: conocido es el Unshielded twisted pair (UTP) se utiliza habitualmente para redes de computadoras confinadas dentro de un edificio, es decir, para redes LAN.

El cable coaxial: puede ser usado como un medio compartido, es decir, una serie de sistemas terminales pueden estar conectados directamente al cable, recibiendo todos ellos lo que envíen los otros sistemas terminales.

La fibra óptica: traslada pulsos de luz, pero los dispositivos ópticos son muy costosos. Representando cada pulso un bit. Un único cable de fibra óptica puede soportar velocidades de bit tremendamente altas, de hasta decenas o incluso centenares de gigabits por segundo. La fibra óptica es inmune a las interferencias electromagnéticas.

Los canales de transmisión radial: transportan señales dentro del espectro electromagnético. Constituyen un medio atractivo, porque no requieren la instalación de cables físicos, pueden atravesar las paredes, proporcionan conectividad a los usuarios móviles y potencialmente pueden transportar una señal a grandes distancias.
Sufren mayor interferencia y tienen tres categorías:

- distancia corta;
- locales y;
- largo alcance (kilómetros).

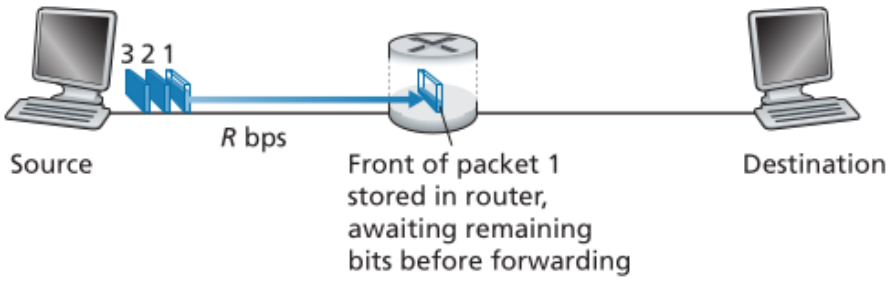
Canales de transmisión satelital: transmiten/reciben microondas en bandas de frecuencia determinadas, los satélites pueden ser geoestacionarios u orbitar más cerca de la tierra.

The Network Core

endsystems intercambian mensajes. Estos mensajes se dividen en **packets** para ser enviados. Cada **packet** viaja a través de **communication links** y **packet switches** (routers or link-layer switches). Se transmite al rate del communication link. Entonces si el rate es R y el largo del link es L, el tiempo de transmisión es:

L/R segundos.

Los **packet switches** usan **store-and-forward transmission**. Debe recibir el packet entero antes de empezar a transmitirlo. Por esta razón, el router habrá transmitido todo el paquete y este habrá sido recibido en su totalidad por el destino. Por tanto, el retardo total es de 2L/R. Si, en lugar de comportarse así, el router reenviará los bits a medida que van llegando (sin esperar a recibir primero todo el paquete), entonces el retardo total sería L/R, puesto que los bits no se verían retenidos en el router. Pero, los routers necesitan recibir, almacenar y procesar el paquete completo antes de reenviarlo.

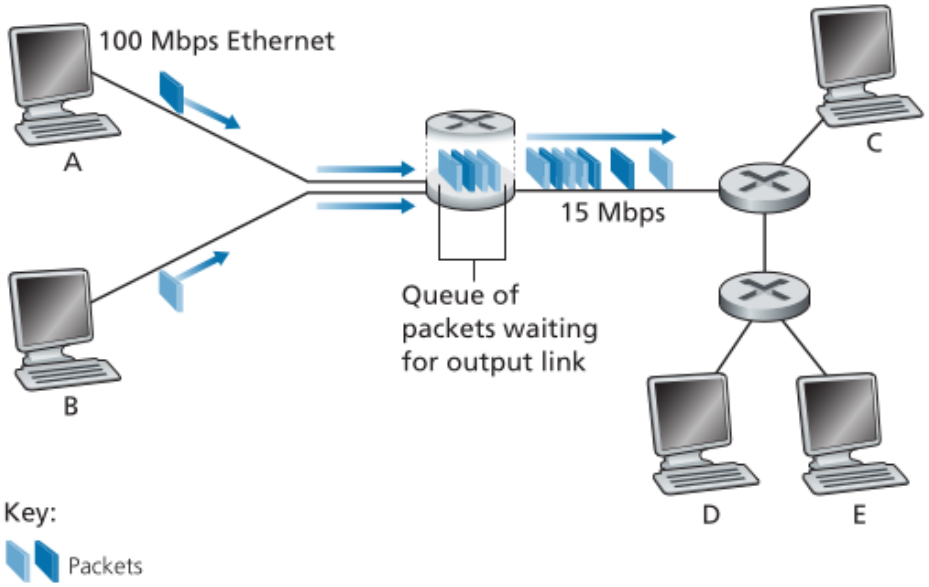


Consideremos ahora el caso general consistente en enviar un paquete desde el origen al destino a través de una ruta compuesta por N enlaces, cada uno de ellos de velocidad R (y en la que, por tanto, hay N-1 routers entre el origen y el destino). Aplicando la misma lógica anterior, vemos que el retardo extremo a extremo es:

$$d_{\text{end-to-end}} = N\frac{L}{R}$$

Queuing Delays and Packet Loss

Por cada link que tiene el packet switch, va a tener un output buffer o output queue donde guarda packets que está a punto de enviar. Puede ocurrir que llegue un packet y deba esperar en el output buffer a que termine otro: queuing delay. Este delay depende de que tan congestionada esté la red.



Si el packet llega y la cola está llena entonces ocurre una packet loss. O el que llega es dropeado o uno que ya está en la cola.

Forwarding Tables and Routing Protocols

Puede pasar que el packet vaya de router en router hasta llegar a su destino, como un conductor perdido sin un mapa que va preguntando a persona en persona hasta llegar a su destino.
¿Cómo determina el router a donde enviar el packet?

Cada sistema tiene su dirección IP. En el header del packet va a figurar la ip de destino. Cada router examina esta porción y se basa en una forwarding table para direccionar el packet. La tabla está creada en base a un routing protocol, por ejemplo el del camino más corto para determinar el próximo router.

Circuit Switching

También puede moverse data de otra manera que no es packet switching, sino circuit switching. En estos casos el path entre end systems se encuentra reservado por el tiempo que dure la comunicación, no ocurre esto en el caso previamente visto. Cuando dos host se quieren comunicar se establece una end-to-end connection, reservando el circuito que los une. Limita la cantidad de conexiones en simultáneo que se pueden tener.

En el caso de packet switching no se garantiza la llegada a destino del mismo, si que se dará el mayor esfuerzo.

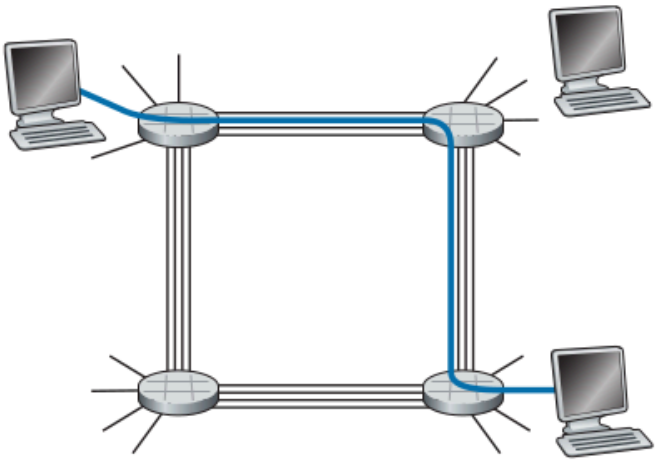


Figure 1.13 ♦ A simple circuit-switched network consisting of four switches and four links

La Figura 1.13 ilustra una red de conmutación de circuitos. En esta red, los cuatro conmutadores de circuitos están interconectados mediante cuatro enlaces. Cada uno de los enlaces tiene cuatro circuitos, por lo que cada enlace puede dar soporte a cuatro conexiones simultáneas. Cada uno de los hosts (por ejemplo, PCs y estaciones de trabajo) está conectado directamente a uno de los conmutadores. Cuando dos hosts desean comunicarse, la red establece una conexión extremo a extremo dedicada entre ellos. Por tanto, para que el host A se comunique con el host B, la red tiene que reservar en primer lugar un circuito en cada uno de los dos enlaces. En este ejemplo, la conexión extremo a extremo dedicada usa el segundo circuito en el primer enlace y el cuarto circuito en el segundo. Dado que cada enlace tiene cuatro circuitos, para cada enlace utilizado por la conexión extremo a extremo, esa conexión obtiene un cuarto de la capacidad total de transmisión del enlace mientras dure la conexión. Así, por ejemplo, si cada enlace entre dos conmutadores adyacentes tiene una velocidad de transmisión de 1 Mbps, entonces cada conexión extremo a extremo de conmutación de circuitos obtiene 250 kbps de velocidad de transmisión dedicada.

Multiplexing in Circuit-Switched Networks

Un circuito es implementado con una frequency-division multiplexing (FDM) o con una time-division multiplexing (TDM). En la primera se le dedica una frecuencia a cada conexión. En el caso de TDM se le otorga a cada conexión una cantidad fija de tiempo.

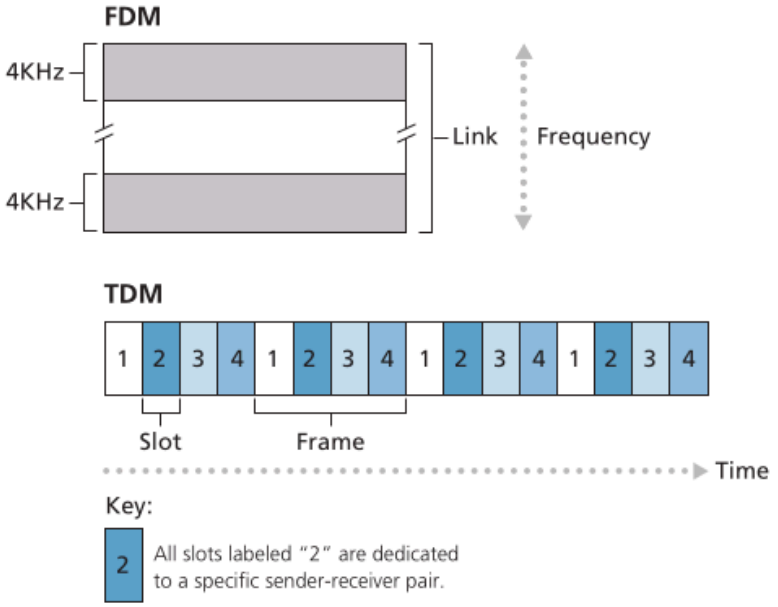


Figure 1.14 ♦ With FDM, each circuit continuously gets a fraction of the bandwidth. With TDM, each circuit gets all of the bandwidth periodically during brief intervals of time (that is, during slots)

Packet Switching Versus Circuit Switching

La **conmutación de circuitos** preasigna el uso del enlace de transmisión independientemente de la demanda, con lo que el tiempo de enlace asignado, pero innecesario, se desperdicia. Por el contrario, la **conmutación de paquetes** asigna el uso del enlace bajo demanda. La capacidad de transmisión del enlace se compartirá paquete a paquete solo entre aquellos usuarios que tengan paquetes que transmitir a través del enlace. La **conmutación de circuitos** es más costosa y por eso (y por el mayor provecho) es que de a poco, todos están migrando a la **conmutación de paquetes**. Básicamente, lo único que te asegura la conmutación de circuitos es que no se pierdan los paquetes.
Ejemplo: Suponga que hay 10 usuarios y que de repente un usuario genera 1.000 paquetes de 1.000 bits, mientras que los usuarios restantes permanecen inactivos y no generan paquetes. Con la tecnología de conmutación de circuitos con multiplexación TDM con 10 particiones por marco y con cada partición formada por 1.000 bits, el usuario activo solo puede emplear su partición por marco para transmitir los datos, mientras que las restantes nueve particiones de cada marco permanecen inactivas. Transcurrirán 10 segundos antes de que el millón de bits de datos del usuario activo hayan sido transmitidos. Sin embargo, con la conmutación de paquetes, el usuario activo puede enviar de forma continuada sus paquetes a la velocidad máxima del enlace de 1 Mbps, ya que no hay ningún otro usuario generando paquetes que tengan que ser multiplexados con los paquetes del usuario activo. En este caso, todos los datos del usuario activo se transmitirán en un segundo.

A Network of Networks

Para conectar ISPs se hace una red de redes. El access ISP es customer, mientras que el global transit ISP es un proveedor. Cada access ISP paga al regional ISP al que se conecta, quien a su vez paga al tier-1 ISP al cual se conecta. Este último no paga a nadie ya que no se conecta a nadie.

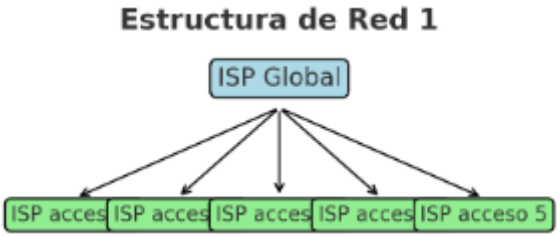
Además en la red hay PoP (grupo de routers en la red del proveedor donde customer ISPs se pueden conectar a un provider ISP. Para reducir costos, customer ISPs que sean vecinos pueden unirse (peer) siendo beneficioso para ambos. También existen Internet Exchange Points (IXP) donde múltiples ISPs pueden unirse (peer). Finalmente tenemos en la última capa los content-provider networks, por ejemplo google.

Existen distintos niveles/estructuras de Red, de los cuales son:

Red 1:

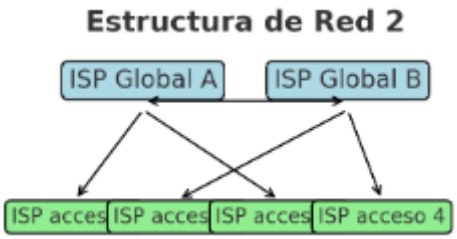
- Todos los **ISP de acceso** (los que conectan directamente a usuarios finales: hogares, empresas, universidades, etc.) se interconectan a través de un **único ISP de tránsito global**.
- Este ISP global tiene routers distribuidos por todo el mundo y conecta a todos los ISPs de acceso.
- Los ISPs de acceso pagan al ISP global → relación **cliente-proveedor**.

Es una visión simplificada, pero poco realista por el costo enorme de mantener una red global única.



Red 2:

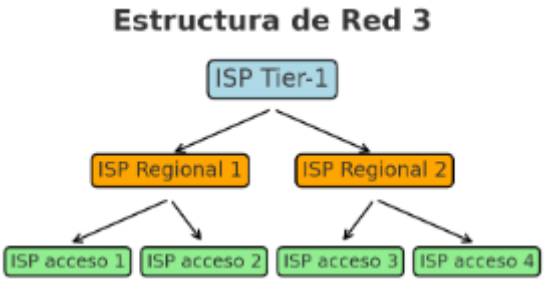
- Existen **múltiples ISPs de tránsito global** en vez de uno solo.
- Los ISPs de acceso pueden elegir con qué ISP de tránsito conectarse, comparando precios y servicios.
- Los ISPs globales compiten entre sí, pero también deben **interconectarse entre ellos**, para garantizar que los clientes de diferentes ISPs de tránsito puedan comunicarse.
  Aquí se establece una **jerarquía de dos niveles**:
  - Arriba → ISPs globales.
  - Abajo → ISPs de acceso.



Red 3:

- Se introduce un nivel intermedio de **ISPs regionales** entre los ISPs de acceso y los ISPs de nivel 1 (tier-1).
- Los **ISPs de acceso** pagan a los **ISPs regionales**, y estos a su vez pagan a los **tier-1**.
- Ejemplo: En China, los ISPs locales → provinciales → nacionales → tier-1.
- Los **tier-1 ISPs** están en la cima de la jerarquía y **no pagan a nadie**, solo se interconectan entre ellos (peering).

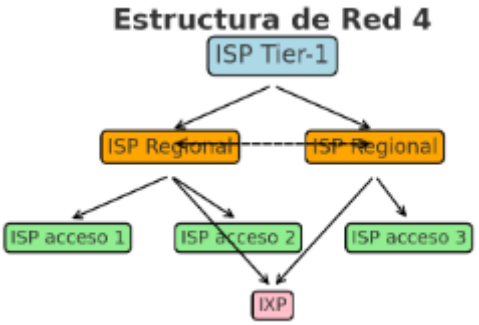
Es una **jerarquía multinivel** que refleja más la realidad del Internet global.



Red 4:

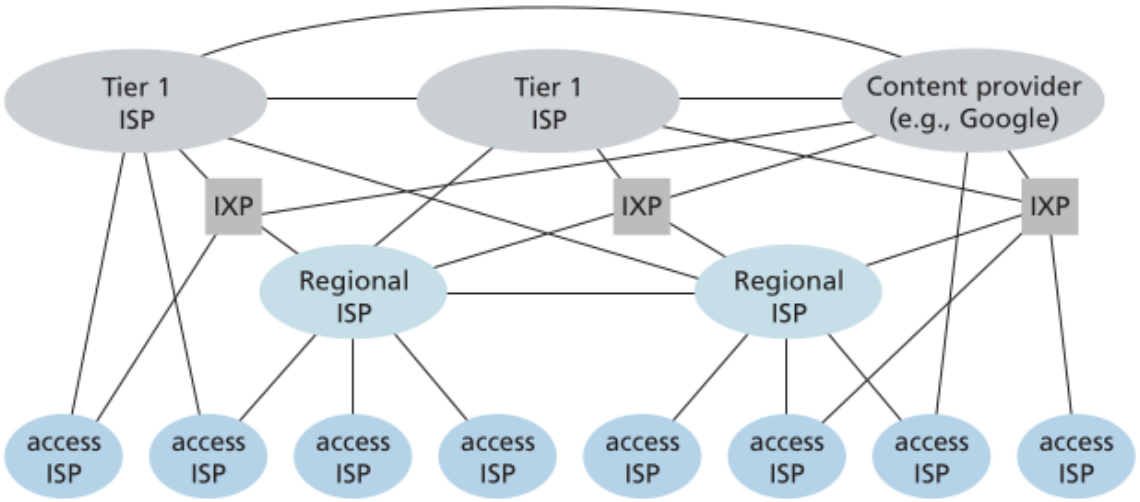
- Se agregan mecanismos y estructuras que hacen la red más robusta y eficiente:
  - **PoPs (Points of Presence):** puntos donde un ISP permite que otros se conecten.
  - **Multi-homing:** un ISP se conecta a varios proveedores para asegurar redundancia.
  - **Peering:** dos ISPs se interconectan directamente (sin pagar) para intercambiar tráfico.
  - **IXPs (Internet Exchange Points):** centros neutrales donde varios ISPs pueden interconectarse.

Aquí aparece el **ecosistema real de interconexiones**, con flexibilidad, redundancia y acuerdos de peering.



Red 5:

- Se incorpora un nuevo actor: las **redes de proveedores de contenido** (ej. Google, Facebook, Netflix).
- Estos proveedores construyen sus **propias redes privadas globales** con centros de datos interconectados.
- Intentan **evitar el tránsito por ISPs de nivel superior** conectándose directamente a ISPs de acceso o regionales, a menudo en IXPs.
- Sin embargo, para llegar a todos los usuarios, siguen necesitando en algunos casos conectarse a tier-1 ISPs (pagando).
  Esto les permite:
  - Reducir costos de tránsito.
  - Mejorar la calidad y control de la entrega de sus servicios.
  - Tener presencia más cercana a los usuarios finales.



Overview of Delay in Packet-Switched Networks

Cuando un paquete viaja de un nodo (host o router) al siguiente nodo (host o router) a lo largo de esa ruta, el paquete sufre varios tipos de retardos en cada uno de los nodos de dicha ruta. Los más importantes de estos retardos son: el **retardo de procesamiento nodal**, el **retardo de cola**, el **retardo de transmisión** y el **retardo de propagación**; todos estos retardos se suman para proporcionar el **retardo nodal total**.

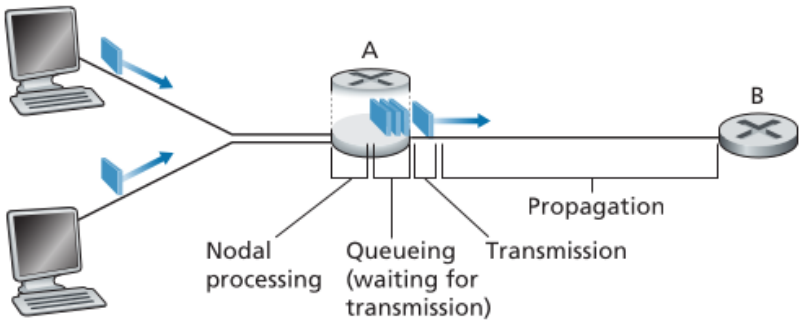


Figure 1.16 ♦ The nodal delay at router A

Un paquete puede transmitirse a través de un enlace solo si actualmente no se está transmitiendo ningún otro paquete a través de él y si no hay otros paquetes que le preceden en la cola; si el enlace está ocupado actualmente o si existen otros paquetes en la cola esperando para ese enlace, entonces el paquete recién llegado tendrá que ponerse a la cola.

Retardo de procesamiento (Processing Delay)

El tiempo requerido para examinar la cabecera del paquete y determinar dónde hay que enviarlo es parte del **retardo de procesamiento**. Una vez efectuado el procesamiento nodal, el router dirige el paquete a la cola situada antes del enlace que lleva al router B.

Retardo de cola (Queuing Delay)

En la cola, el paquete experimenta un retardo de cola mientras espera para ser transmitido a través del enlace. La duración del retardo de cola para un determinado paquete dependerá del número de paquetes que hayan llegado antes a la cola y que estén esperando para ser transmitidos por el enlace. Si la cola está vacía y no se está transmitiendo ningún paquete actualmente, entonces el retardo de cola de nuestro paquete será cero. Por el contrario, si hay mucho tráfico y muchos paquetes están esperando también para ser transmitidos, el retardo de cola será grande. En la práctica, los retardos de cola pueden ser del orden de microsegundos a milisegundos.

Retardo de transmisión (Transmission Delay)

Suponiendo que los paquetes se transmiten de manera que el primero que llega es el primero que sale, como suele ser común en las redes de conmutación de paquetes, nuestro paquete sólo puede ser transmitido después de que todos los paquetes que hayan llegado antes que él hayan sido transmitidos. Sea la longitud del paquete igual a **L bits** y la velocidad de transmisión del enlace del router A hasta el router B igual a **R bits/segundo**. **Por ejemplo**, para un enlace Ethernet a 10 Mbps, la velocidad es R = 10 Mbps; para un enlace Ethernet a 100 Mbps, la velocidad será R = 100 Mbps. **El retardo de transmisión será igual a L/R**. Este es el tiempo necesario para introducir (es decir, transmitir) todos los bits del paquete en el enlace. Normalmente, en la práctica, los retardos de transmisión son del orden de microsegundos a milisegundos.

Retardo de propagación (Propagation Delay)

Una vez que un bit ha entrado en el enlace, tiene que propagarse hasta el router B. El tiempo necesario para propagarse desde el principio del enlace hasta el router B es el retardo de propagación. El bit se propaga a la velocidad de propagación del enlace, que depende del medio físico del enlace (es decir, de que el medio sea fibra óptica, cable de cobre de par trenzado, etc.) y está comprendido en el rango entre:

$$2 \cdot 10^8 \text{ m/s y } 3 \cdot 10^8 \text{ m/s}$$

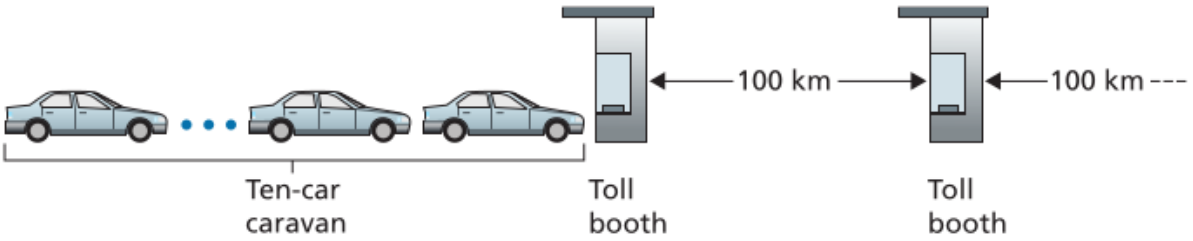
que es igual, o ligeramente inferior, a la velocidad de la luz. El retardo de propagación es igual a la distancia entre dos routers dividida por la velocidad de propagación. Es decir, el retardo de propagación es igual a **d/s**, donde **d** es la **distancia** entre el **router A** y el **router B** y **s** es la **velocidad** de propagación del enlace. Una vez que el último bit del paquete se

ha propagado hasta el nodo B, este y todos los bits anteriores del paquete se almacenan en el router B.

Comparing Transmission and Propagation Delay

El retardo de transmisión es la cantidad de tiempo necesario para que el router saque fuera el paquete; es una función de la longitud del paquete y de la velocidad de transmisión del enlace, pero no tiene nada que ver con la distancia existente entre los dos routers. Por el contrario, el retardo de propagación es el tiempo que tarda un bit en propagarse de un router al siguiente; es una función de la distancia entre los dos routers, pero no tiene nada que ver con la longitud del paquete ni con la velocidad de transmisión del enlace.

Ejemplo: El tiempo necesario para que el peaje deje pasar a la caravana completa hacia el siguiente tramo de autopista es igual a (10 coches)/(5 coches/minuto) = 2 minutos. Este tiempo es análogo al **retardo de transmisión** de un router. El tiempo necesario para que un coche se desplace desde la salida del peaje hasta el siguiente puesto de peaje es 100 km/(100 km/hora) = 1 hora. Este tiempo es análogo al **retardo de propagación**. Por tanto, el tiempo que transcurre desde que la caravana queda colocada delante de un peaje hasta que vuelve a quedar colocada delante del siguiente peaje es la **suma del tiempo de transmisión y el tiempo de propagación** (en este caso, dicho tiempo será igual a 62 minutos).



Algo que puede pasar también es: los primeros bits de un paquete pueden llegar a un router mientras que muchos de los bits restantes del paquete todavía están esperando a ser transmitidos por el router anterior.

Sean d_{proc}, d_{cola}, d_{trans} y d_{prop} los retardos de procesamiento, de cola, de transmisión y de propagación, respectivamente. Entonces el retardo nodal total estará dado por:

$$d_{nodal} = d_{proc} + d_{cola} + d_{trans} + d_{prop}$$

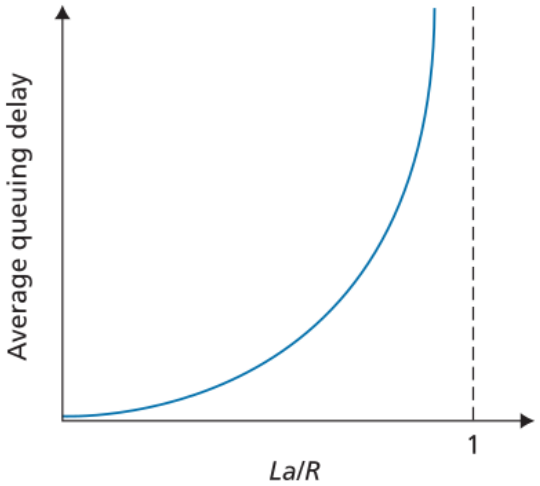
En la cátedra utilizamos "**t**" en vez de "**d**".

Las contribuciones de estos componentes de retardo pueden variar significativamente. Por ejemplo, $\mathbf{d_{prop}}$ **puede ser despreciable** (digamos que un par de microsegundos) para un enlace que conecte dos routers del mismo campus universitario; sin embargo, $\mathbf{d_{prop}}$ será de cientos de milisegundos para dos routers interconectados mediante un enlace vía satélite geoestacionario y puede llegar a ser el término dominante en $\mathbf{d_{nodal}}$. Del mismo modo, $\mathbf{d_{trans}}$ **puede ser despreciable o significativo**. Su contribución **normalmente es despreciable** para velocidades de transmisión de 10 Mbps y superiores (por ejemplo, **para las redes LAN**); sin embargo, puede ser igual a cientos de milisegundos para paquetes grandes de Internet enviados a través de enlaces de acceso telefónico que usen módems de baja

velocidad. El retardo de procesamiento, $\mathbf{d_{proc}}$, **suele ser despreciable**; sin embargo, tiene una **gran influencia sobre la tasa de transferencia máxima del router**, que es la velocidad máxima a la que un router puede reenviar los paquetes.

Queuing Delay and Packet Loss

Traffic intensity: L*a/R, donde **L** es la cantidad de bits [bits], **a** es [packets/sec] y **R** es [bits/sec]. Si el resultado es mayor a 1, se dice que la tasa promedio a la que arriban los bits, excede la tasa promedio de inserción en el medio, en tal caso la cola del buffer va a crecer indefinidamente y el tiempo de demora en el buffer va a tender a infinito.



Si la intensidad de tráfico es próxima a cero, entonces las llegadas de paquetes serán pocas y estarán bastante espaciadas entre sí, por lo que será improbable que un paquete que llegue a la cola se encuentre con que hay otro paquete en la cola. Por tanto, el retardo medio de cola será próximo a cero. Por el contrario, cuando la intensidad de tráfico es próxima a 1, habrá intervalos de tiempo en los que la velocidad de llegada exceda a la capacidad de transmisión, por lo que se formará una cola durante estos periodos de tiempo.

El **porcentaje** de **paquetes perdidos** aumenta a medida que crece la intensidad de tráfico. Por tanto, el rendimiento de un **nodo** suele **medirse** no sólo en función del retardo, sino también en función de la **probabilidad de pérdida de paquetes**. Esto se ve cuando la **intensidad de tráfico** es **mayor** a **1**.

End-to-End Delay

Suponiendo que hay N-1 routers entre el host origen y host destino, la red no está congestionada (los retardos de cola son despreciables), que el tiempo de procesamiento en cada router y del host origen es $\mathbf{d_{proc}}$, que la velocidad de transmisión de salida de cada router y del host de origen es de R bits/segundo y que el retardo de propagación en cada enlace es igual a dprop. Los retardos nodales se suman para proporcionar el retardo extremo a extremo

$$d_{\text{end}-\text{end}} = N\,(d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}})$$

Throughput in Computer Networks

Considerando la transferencia de un archivo de gran tamaño desde el host A al host B a través de una red.

La **tasa de transferencia** instantánea en cualquier instante de tiempo es la velocidad (en bits/segundo) a la que el host B recibe el archivo. Si el archivo consta de F bits y la transferencia dura T segundos hasta que el host B recibe los F bits, entonces la **tasa media de transferencia** del archivo es igual a F/T bits/segundo.

En este análisis descartamos cualquier retardo extra de protocolos.

Throughput: Es la efectividad con la que transmito datos. Un ejemplo, si un auto tiene la capacidad de ir a 200km/h pero la autopista está llena, no irá a 200km/h. Se mide en bits/segundos.

Ancho de banda: Es la cantidad de datos capaz de transmitir, sería el Throughput máximo.

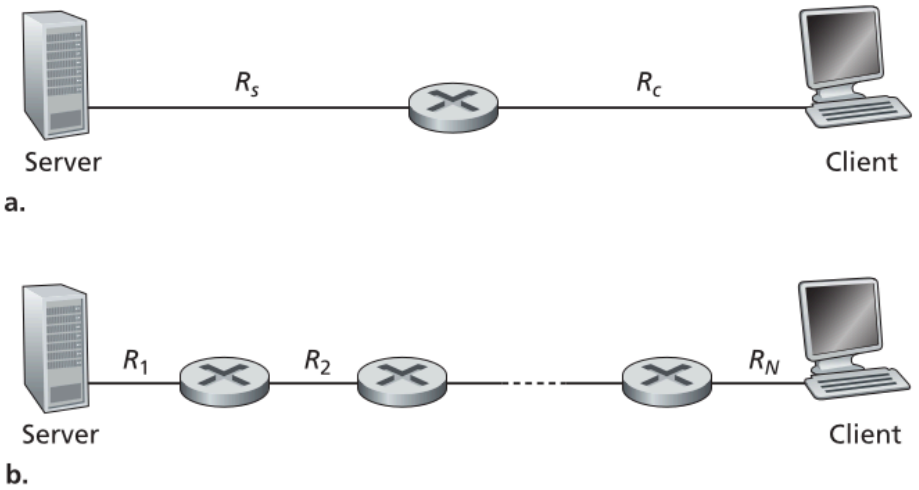


Figure 1.19 ♦ Throughput for a file transfer from server to client

Con el **Ejemplo 1.19(a):** El servidor no puede bombear los bits a través de su enlace a una velocidad mayor que Rs bps; y el router no puede reenviar los bits a una velocidad mayor que Rc bps.

- Si **Rs < Rc**, entonces los bits bombeados por el servidor "fluirán" a través del router y llegarán al cliente a una velocidad de Rs bps, obteniéndose una tasa de transferencia de Rs bps.
- Si, por el contrario, **Rc < Rs**, entonces el router no podrá reenviar los bits tan rápidamente como los recibe. En este caso, los bits abandonarán el router a una velocidad de solo Rc, dando lugar a una tasa de transferencia de extremo a extremo igual a Rc.

(Observe también que si continúan llegando bits al router a una velocidad Rs, y siguen abandonando el router a una velocidad igual a Rc, la cantidad de bits en el router que están esperando a ser transmitidos hacia el cliente aumentará constantemente, lo que es una situación nada deseable.) Por tanto, en esta sencilla red de dos enlaces, la **tasa de transferencia** es **mín{Rc, Rs}**.

Una vez determinada la **tasa de transferencia**, podemos **aproximar el tiempo** que se tardará en **transferir un archivo** de gran tamaño de **F** bits desde el servidor al cliente, mediante la fórmula **F/mín{Rs, Rc}**.

Con el **Ejemplo 1.19(b):** Aplicando el mismo análisis que para la red de dos enlaces, podemos determinar que la tasa de transferencia para enviar un archivo desde el servidor al cliente es mín{R1, R2,...,RN}, que es de nuevo la velocidad de transmisión del enlace cuello de botella existente en la ruta entre el servidor y el cliente.

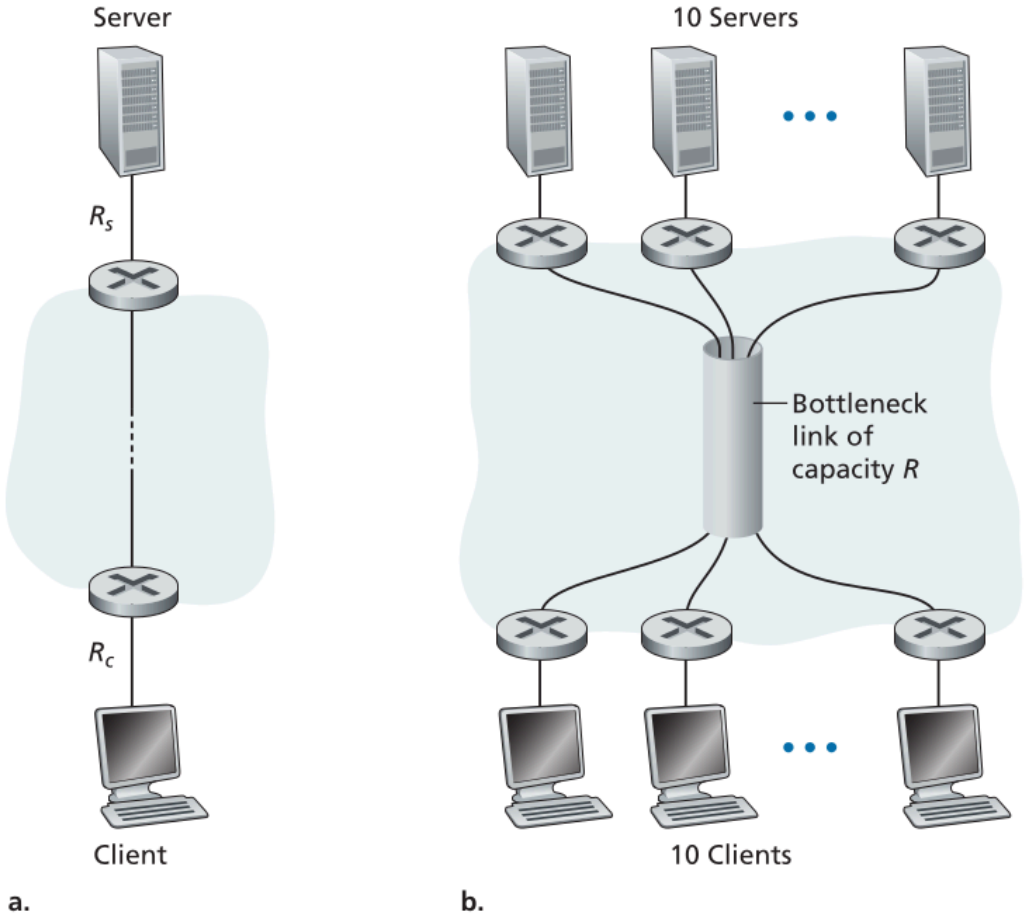


Figure 1.20 ♦ End-to-end throughput: (a) Client downloads a file from server; (b) 10 clients downloading with 10 servers

Para el **Ejemplo 1.20 (a):** Suponiendo ahora que todos los enlaces existentes en el núcleo de la red de comunicaciones tienen velocidades de transmisión muy altas, muy por encima de Rs y Rc. Dado que el núcleo de la red es como una tubería ancha en este ejemplo, la velocidad a la que los bits pueden fluir desde el origen hasta el destino es, de nuevo, el mínimo de Rs y Rc, es decir, tasa de transferencia = mín{Rs, Rc}. Por tanto, en la actualidad, el factor de restricción de la tasa de transferencia en Internet es, normalmente, la red de acceso.

Para el **Ejemplo 1.20 (b):** Hay 10 pares cliente-servidor, supongamos que estas 10 descargas son el único tráfico existente en la red en este momento. Supongamos que los enlaces de acceso de todos los servidores tienen la misma velocidad Rs, que los enlaces de acceso de todos los clientes tienen la misma velocidad Rc y que las velocidades de transmisión de todos los enlaces del núcleo (excepto el enlace común de velocidad R) son mucho mayores que Rs, Rc y R.

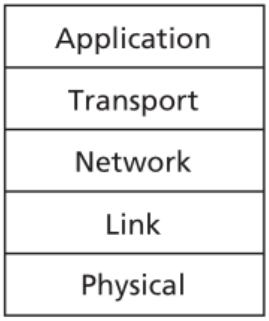
Si la velocidad del enlace común R es grande, entonces la tasa de transferencia será de nuevo, mín{Rs, Rc}. **¿Qué ocurre si la velocidad del enlace común es del mismo orden que Rs y Rc?** ⇒

Supongamos que **Rs = 2 Mbps, Rc = 1 Mbps, R = 5 Mbps** y que el enlace común divide su velocidad de transmisión en partes iguales entre las 10 descargas. Entonces, el **cuello de botella** para cada descarga ya no se encuentra en la red de acceso, sino en el **enlace compartido del núcleo**, que sólo proporciona una tasa de transferencia de 500 kbps a cada descarga. Luego la **tasa de transferencia extremo a extremo** para cada descarga ahora se habrá reducido a 500 kbps.

Con esto vemos que un enlace con una velocidad de transmisión alta puede ser, de todos modos, el enlace cuello de botella para la transferencia de un archivo si hay muchos otros flujos de datos atravesando también ese enlace.

Protocol Layering

Los protocolos de redes están organizados en capas. Cada protocolo pertenece a una capa. Cada capa provee de un servicio a la capa superior.



Five-layer Internet protocol stack

Figure 1.23 ♦ The Internet protocol stack

La **capa de aplicaciones** es donde residen las redes de aplicaciones y sus protocolos. Por ejemplo, HTTP (que permite la solicitud y transferencia de documentos web), SMTP (que permite la transferencia de mensajes de correo electrónico), VPN, DNS, FTP (que permite la transferencia de archivos entre dos sistemas terminales), SSH, DHCP, LDAP. Se distribuye en múltiples hosts tal que intercambian mensajes entre ellos usando algún protocolo. A este paquete de información de la capa de aplicación lo denominaremos **mensaje**.

La **capa de transporte** dice es la que lleva esos mensajes de una aplicación a otra. Existen dos protocolos: TCP y UDP. Nos referimos a un paquete de capa de transporte como un **segmento**.

La **capa de red** provee el servicio que **entrega un segmento a la capa de transporte destino**. El protocolo de la capa de transporte Internet (TCP o UDP) de un host de origen pasa un segmento de la capa de transporte y una dirección de destino a la capa de red, al igual que nosotros damos al servicio de correo postal una carta con una dirección de destino. Luego, la capa de red proporciona el servicio de suministrar el segmento a la capa de transporte del host de destino. Existe un único protocolo: IP.

La **capa de enlace** mueve paquetes de un nodo (host o router) a otro. Ejemplos: ethernet, wi-fi, bluetooth, NFC, 4G, cable modem, ADSL.

La **capa física** es la que mueve cada bit a través del enlace. Ejemplos: fibra óptica, cable coaxial, UTP.

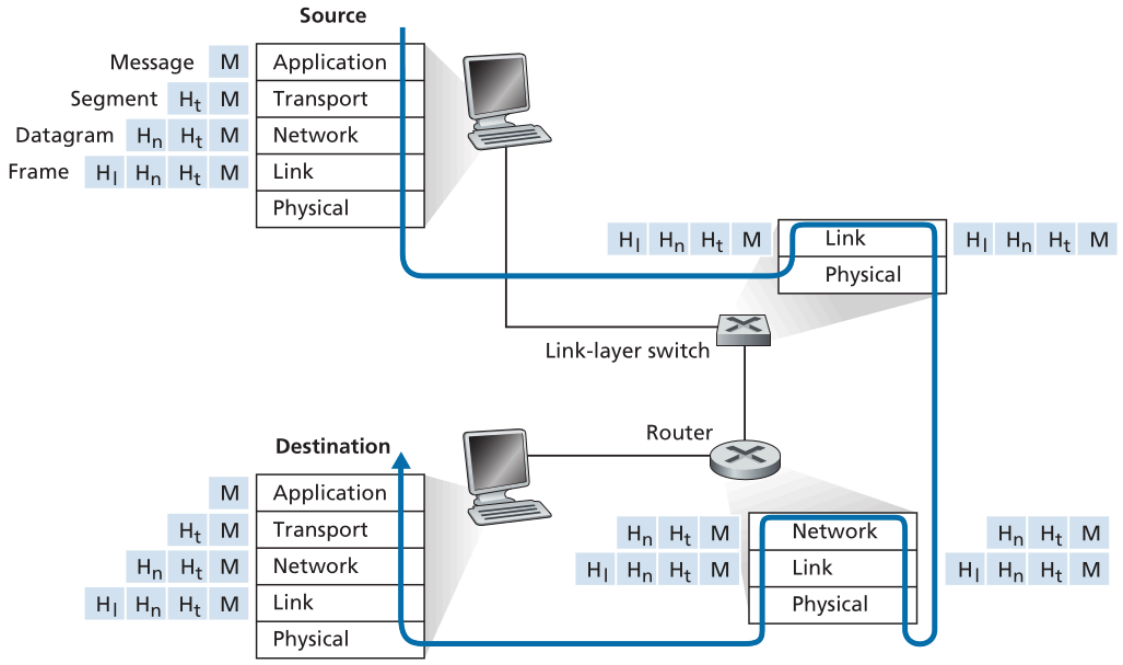


Figure 1.24 ♦ Hosts, routers, and link-layer switches; each contains a different set of layers, reflecting their differences in functionality

La **Figura 1.24** muestra cómo viajan los datos en la red a través de las capas del modelo.

- **Hosts (emisor y receptor)**: implementan las 5 capas (Aplicación, Transporte, Red, Enlace y Física).
- **Switch de capa de enlace**: solo usa capas 1 y 2 (Física y Enlace). No entiende direcciones IP, pero sí direcciones MAC. Sirve para trabajar con end systems dentro de una misma red.
- **Router**: usa capas 1 a 3 (Física, Enlace y Red). Puede manejar IP y enrutar paquetes. Sirve para trabajar con end systems en otras redes.

Los datos viajan encapsulados:

- La **Aplicación** genera un **mensaje (M)**.
- La **Transporte** agrega su cabecera → **segmento (Ht + M)**.
- La **Red** agrega su cabecera → **datagrama (Hn + Ht + M)**.
- El **Enlace** agrega su cabecera → **trama (Hl + Hn + Ht + M)**.

En el receptor, ocurre lo inverso: cada capa quita su cabecera hasta entregar el mensaje a la aplicación.

La idea central es la **encapsulación**: cada capa agrega su propia información de control, y los dispositivos de red solo procesan las capas que entienden.

Notar que routers y switches de la capa de linkeo no implementan todas las capas. En cada capa un packet tiene dos campos: **header** y **payload**. El último es un packet de la capa anterior y header sirve para que la capa actual actúe. A esto se llama encapsulación.

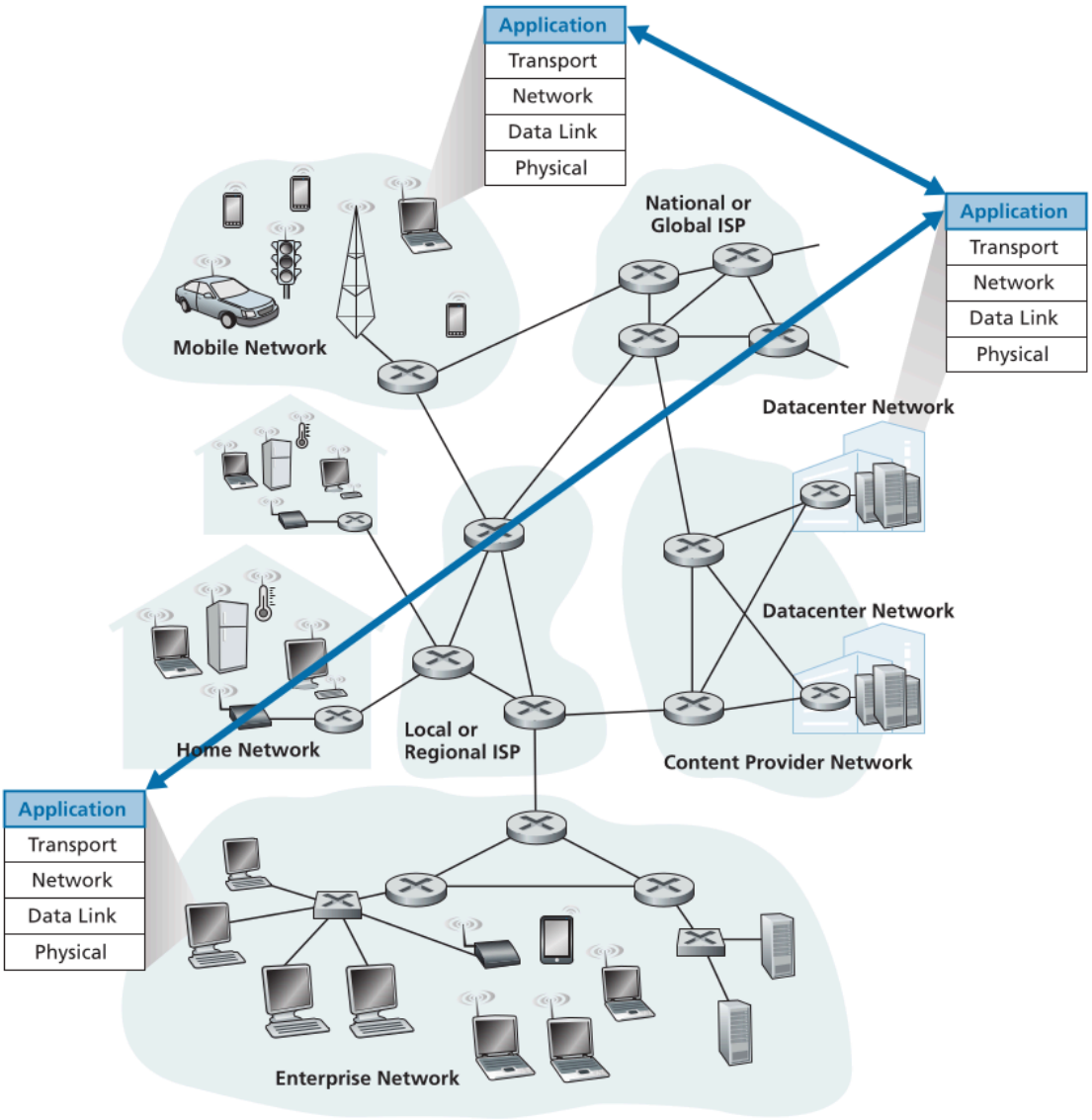
Networks Under Attack

Tipos de ataques de seguridad:

- DoS (denial-of-service) → se divide en tres categorías: vulnerability attack (se aprovecha de vulnerabilidades), connection flooding (abre muchas conexiones, saturando la red), bandwidth flooding (envia muchos paquetes, saturando los enlaces)
- Packet sniffers (Un receptor pasivo que registra una copia de todos los paquetes que pasan por él)
- Masquerade (hacerse pasar por otro usuario, cambiando la dirección, **IP spoofing**)
- Man in the middle

Capítulo 2 - Application Layer

Al desarrollar una aplicación, hay que escribir software de tal manera que pueda correr en múltiples sistemas (no es necesario en routers o link-layer switches, estos no funcionan en la capa de aplicación). La arquitectura de la aplicación está diseñada por el desarrollador, a diferencia de la arquitectura de la red. Esta puede ser cliente-servidor o peer-to-peer.



Network Application Architectures

En una arquitectura **cliente-servidor** hay un host que sirve de servidor, que está siempre encendido, atendiendo solicitudes de otros hosts denominados clientes. Ejemplos: Web, FTP, Telnet, email.

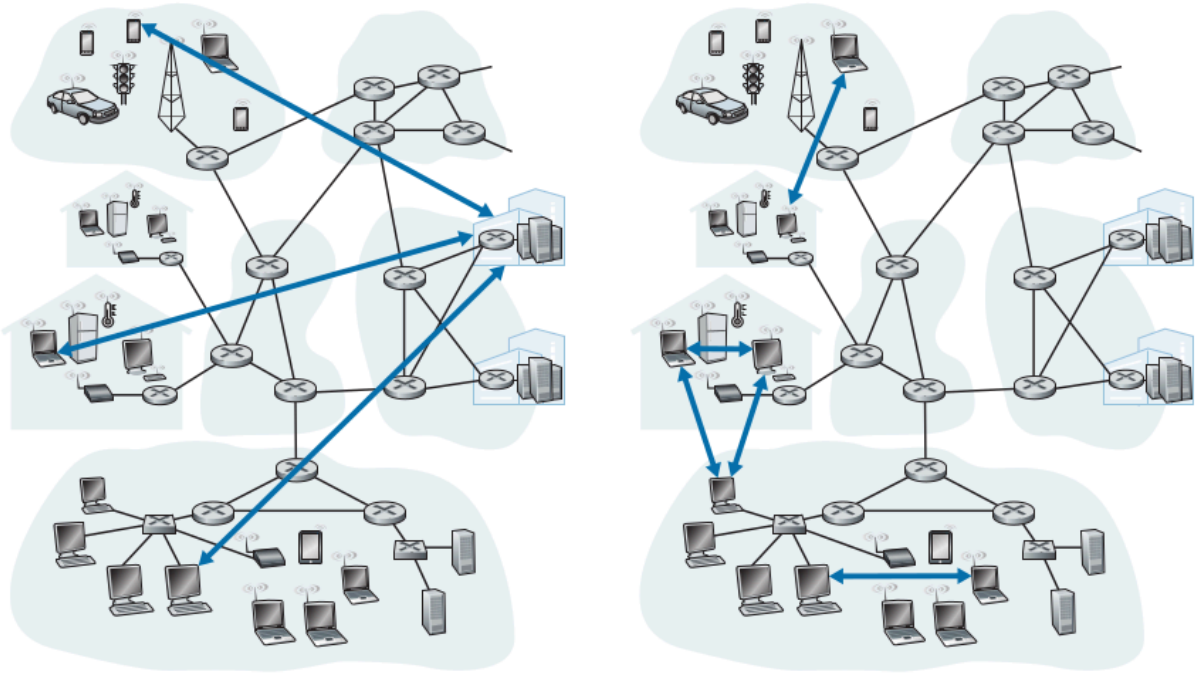
Características de la arquitectura **cliente-servidor**:

- Los clientes no interactúan entre sí, no pueden comunicarse directamente, sino que lo hacen a través del servidor.

- El servidor tiene una dirección IP fija y conocida tal que un cliente siempre puede contactarlo.

En general, en esta arquitectura no alcanza con un solo servidor para atender todas las solicitudes de clientes, por eso existen los data centers.

En una **arquitectura P2P** no se depende de servidores, sino que se basa en la comunicación directa entre pares de hosts (peers). Estos **peers** son los dispositivos controlados por los usuarios finales (smartphones, tablets, notebooks). Ejemplos: BitTorrent, Skype.
Existen aplicaciones que utilizan ambas arquitecturas, por ejemplo en servicios de mensajería donde las **direcciones IP** se trackean mediante servidores pero los **mensajes** son **enviados** directamente de **usuario a usuario**.
Una de las **ventajas** de **P2P** es su **self-scalability**, en el caso de una aplicación de **file-sharing** cada host suma capacidad de servicio a la aplicación al distribuir archivos para otros peers. También se dice que es eficiente económicamente ya que no requiere de una cantidad significativa de servidores y ancho de banda. Las **desventajas** son que tiene **problemas** de **seguridad**, **reliability** y **performance**.



a. Client-server architecture

b. Peer-to-peer architecture

Processes Communicating

Las aplicaciones en la red consisten de pares de procesos que se envían mensajes entre sí a través de la red. Por ejemplo, en la Web un cliente browser intercambia mensajes con el servidor web; en una aplicación de **file-sharing (p2p)**, un archivo es transferido **desde** un proceso en un **peer a otro** proceso en otro **peer**. En este último caso, se dice que **el cliente es el que descarga el archivo** y **el servidor el que lo sube**.

En cualquier comunicación entre un par de procesos, se dice que **el proceso que inicia la comunicación** es el **cliente** y **el que espera a ser contactado** para empezar la comunicación es el **servidor**.

Un proceso envía y recibe mensajes de la red a través de una interfaz llamada socket. El **socket** es la interfaz entre la capa de aplicaciones y la capa de transporte dentro de un host. También se puede pensar como la **API (Application Programming Interface)** entre la **aplicación** y la **red**.

El desarrollador de la aplicación solo tiene control sobre los siguientes aspectos del socket:

- La elección del protocolo de transporte.
- La capacidad de ajustar algunos parámetros de la capa de transporte, como el tamaño del buffer y el tamaño de los segmentos.

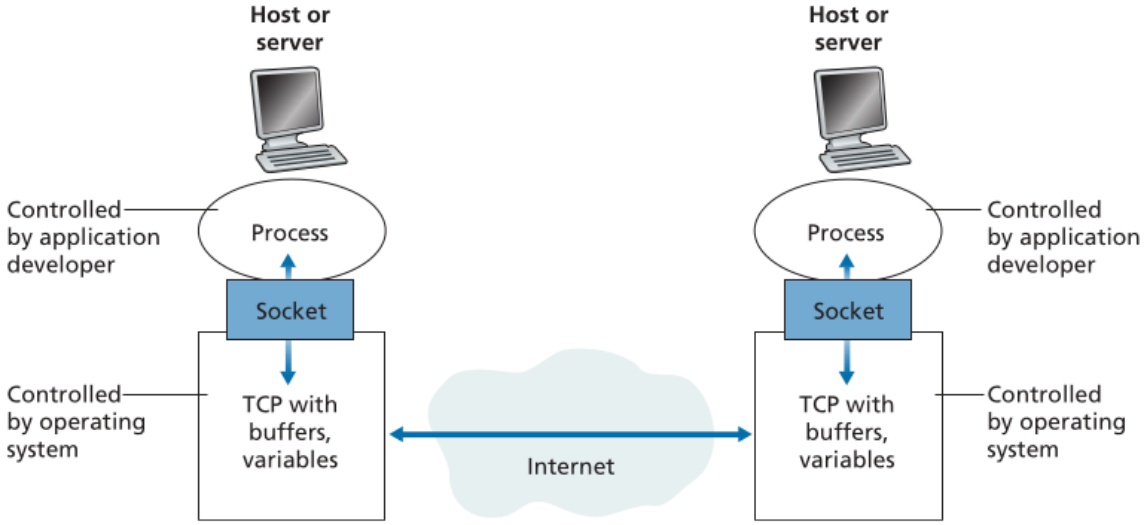


Figure 2.3 ♦ Application processes, sockets, and underlying transport protocol

Para que un proceso pueda enviarle algo a otro proceso, este último tiene que tener una dirección. Para identificar al proceso recibido, se necesita:

- La dirección del host (**IP**) y;
- Un identificador del proceso en el host destino (**puerto**).

Transport Services Available to Applications

La aplicación manda mensajes a través del socket. Del otro lado del socket está el protocolo de la capa de transporte que tiene que hacer llegar esos mensajes al socket del proceso receptor. El desarrollador de la aplicación debe elegir el protocolo que mejor se adapte a sus necesidades.

¿Qué servicios ofrece el protocolo de la capa de transporte a las aplicaciones que lo invocan?

- Transferencia de datos confiable (**reliable data transfer**): Se dice que es confiable cuando puede garantizar que no se pierdan o corrompan paquetes de datos.
- Caudal (**throughput**): Es la velocidad a la que el proceso origen puede entregarle bits al proceso receptor. Como el ancho de banda se comparte con otras sesiones, el rendimiento puede variar. Existen aplicaciones (bandwidth-sensitive applications) que piden garantía de un rendimiento mínimo (por ej. aplicaciones multimedia).

Otras (elastic applications) usan lo que esté disponible (por ej. mail y transferencia de archivos).

- **Timing**: Por ejemplo, se podría garantizar que ningún bit tarde más de X ms en llegar a destino (interactive real-time applications, teleconferencias, juegos multijugador).
- **Seguridad**: Por ejemplo, se podría encriptar toda la data enviada.

Application	Data Loss	Throughput	Time-Sensitive
File transfer/download	No loss	Elastic	No
E-mail	No loss	Elastic	No
Web documents	No loss	Elastic (few kbps)	No
Internet telephony/ Video conferencing	Loss-tolerant	Audio: few kbps–1 Mbps Video: 10 kbps–5 Mbps	Yes: 100s of msec
Streaming stored audio/video	Loss-tolerant	Same as above	Yes: few seconds
Interactive games	Loss-tolerant	Few kbps–10 kbps	Yes: 100s of msec
Smartphone messaging	No loss	Elastic	Yes and no

Figure 2.4 ♦ Requirements of selected network applications

Transport Services Provided by the Internet

Internet provee dos protocolos de transporte: TCP y UDP.

TCP Services

El modelo de servicio TCP de entrada provee dos servicios: **connection-oriented** (establece una conexión entre las dos entidades antes de que empiece el flujo de datos - **handshaking procedure**) y **reliable data transfer**. Además, cuenta con un mecanismo para controlar la congestión. Para proveer seguridad, se lo puede mejorar con SSL en esta capa de aplicación.

UDP Services

El modelo UDP ofrece servicios mínimos. Es **connectionless** (no handshaking), **unreliable data transfer** (hasta podría pasar que los datos lleguen en distinto orden al que se enviaron), no tiene un mecanismo para controlar la congestión.
Observación: hoy en día Internet no da garantías de timing ni rendimiento; sin embargo, suele funcionar bien.

Un protocolo de la **capa de aplicaciones** define cómo intercambiar mensajes entre procesos. En particular:

- Los tipos de mensajes a intercambiar
- La sintaxis de los mensajes
- La información de los campos que se envían en el mensaje
- Reglas para determinar a un proceso cuándo y cómo enviar y recibir mensajes

The Web and HTTP

HTTP es el protocolo de la capa de aplicación de la Web. Se implementa en dos programas: en el **cliente** y en el **servidor**. El programa cliente y el programa servidor interactúan entre sí intercambiando mensajes HTTP. **HTTP define** la **estructura** de los **mensajes** y cómo deben enviarlos tanto cliente como servidor.

Una **página web** consiste de **objetos**. Un objeto en este contexto, es un archivo **accesible** por una **URL** (puede ser una imagen JPEG, un archivo HTML, un applet de Java, etc.). Cada **URL** se compone por dos elementos:

- el **hostname** del servidor que aloja el objeto y;
- el **path** al objeto.

HTTP define cómo los clientes web deben solicitar páginas web al servidor web y cómo los servidores deben enviar dichas páginas al cliente.
HTTP utiliza TCP como protocolo de transporte.

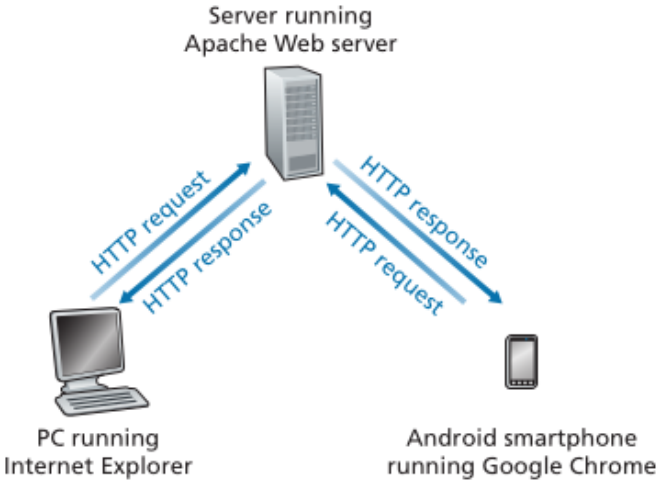


Figure 2.6 ♦ HTTP request-response behavior

Observación: el servidor responde solicitudes sin almacenar ningún tipo de información sobre el cliente → **stateless protocol**. Responde a demanda.

Non-Persistent and Persistent Connections

Conexiones **persistentes** son aquellas donde todas las solicitudes se realizan bajo una misma conexión TCP, si se utilizan conexiones TCP separadas por cada par de **request/response** se dice que es una conexión **no persistente**. Esta es una decisión que debe tomar el desarrollador de la aplicación.
Haciendo una estimación rápida del tiempo total que lleva solicitar un archivo con una conexión, obtenemos 2 RTT (round-trip time - tiempo que tarde en ir y volver). 1 RTT de handshaking y 1 al solicitar y obtener la respuesta.

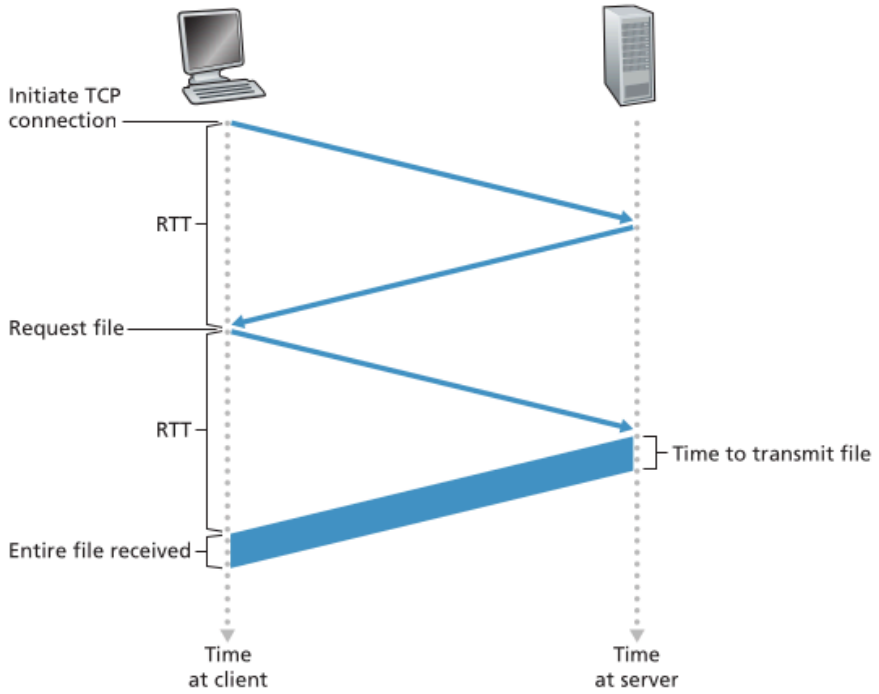


Figure 2.7 ♦ Back-of-the-envelope calculation for the time needed to request and receive an HTML file

Una **desventaja** de las conexiones **no persistentes** es el hecho de que tenga que crearse y mantenerse una nueva conexión por cada objeto solicitado. Por **cada conexión** se **alocan buffers** y **variables**, entonces puede resultar caro. Además, cada objeto tiene una demora de **2 RTTs**.

En conexiones **persistentes**, se deja abierta la conexión y normalmente, el servidor HTTP la cierra luego de cierta cantidad de tiempo sin haber recibido nada.

Un mensaje de solicitud típico:

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
Connection: close
User-agent: Mozilla/5.0
Accept-language: fr
```

La primera línea se denomina **request line**, las siguientes son los **headers**.

- La **request line** se compone por tres partes: el método, la URL y la versión HTTP. El método puede ser GET, PUT, POST, DELETE o HEAD.
- **Host:** indica dónde reside el objeto.
- **Connection**: close indica que debe cerrarse la conexión luego de responder.
- **User-agent:** el browser que está haciendo la solicitud.
- **Accept-language:** indica el idioma preferente del objeto.

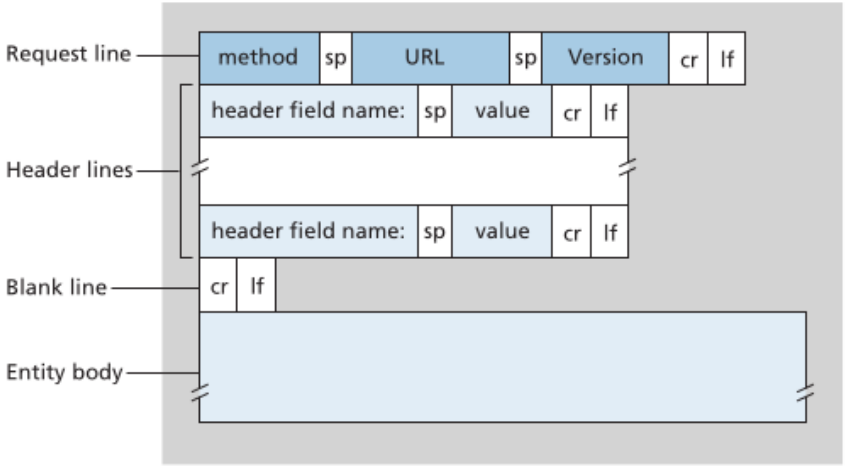


Figure 2.8 ♦ General format of an HTTP request message

Mensaje típico de respuesta:

```
HTTP/1.1 200 OK
Connection: close
Date: Tue, 18 Aug 2015 15:44:04 GMT
Server: Apache/2.2.3 (CentOS)
Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
Content-Length: 6821
Content-Type: text/html

(data data data data data ...)
```

Se compone de: tiene una **status line** (3 campos: versión del protocolo, código de status y el mensaje de status), **seis header lines** y un **entity body** (contiene lo que se pidió en el request).
Las header lines: **date** tiene hora y día en que el servidor envía data, **Server** indica que servidor generó el mensaje, **Last modified** indica el momento de creación o última modificación del objeto, **Content-Lenght** indica el largo del body y **Content-type** el tipo del body (HTML por ej).

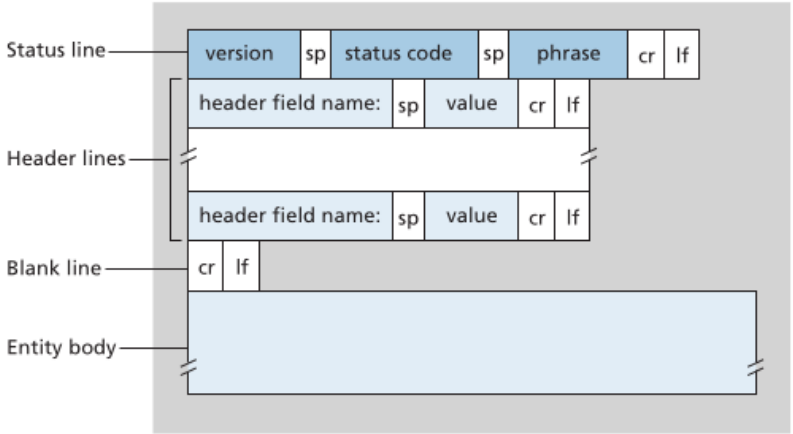


Figure 2.9 ♦ General format of an HTTP response message

Algunos status codes comunes y sus frases incluidas:

- 200 OK
- 301 Moved Permanently: la nueva URL se especifica en el campo Location.

- 400 Bad Request: el servidor no pudo interpretar la solicitud. Suele ser un error del cliente.
- 404 Not Found
- 505 HTTP Version Not Supported

User-Server Interaction: Cookies

Si bien **HTTP** es **stateless**, hay casos donde se **necesita identificar usuarios** ya sea para restringirles acceso o para cambiar el funcionamiento según el tipo de usuario. Para ello, se utilizan las Cookies.

Una **cookie** está **compuesta por**:

- un **header** en el mensaje de **respuesta** HTTP,
- un **header** en el mensaje de **solicitud** HTTP,
- un **archivo** que se **almacena** en el **host** pero lo controla el browser del usuario y una base de datos backend en el sitio web.

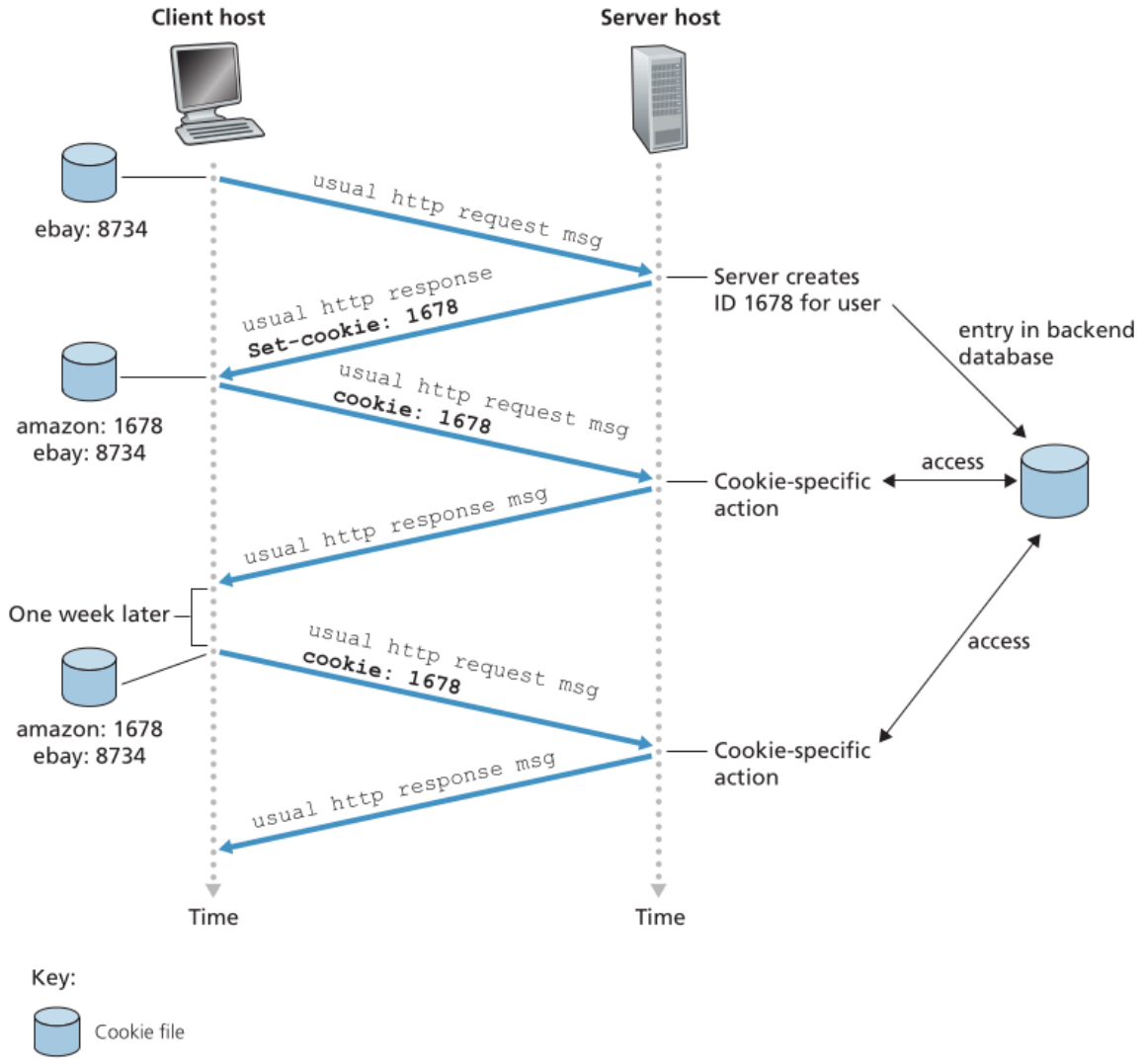


Figure 2.10 ♦ Keeping user state with cookies

Web Caching

Una **caché web** (o servidor proxy) es una entidad de red que satisface solicitudes HTTP en nombre de un servidor web de origen. La caché web tiene su propio disco de almacenamiento donde mantiene copias de los objetos más recientemente consultados. Es configurable desde el browser.

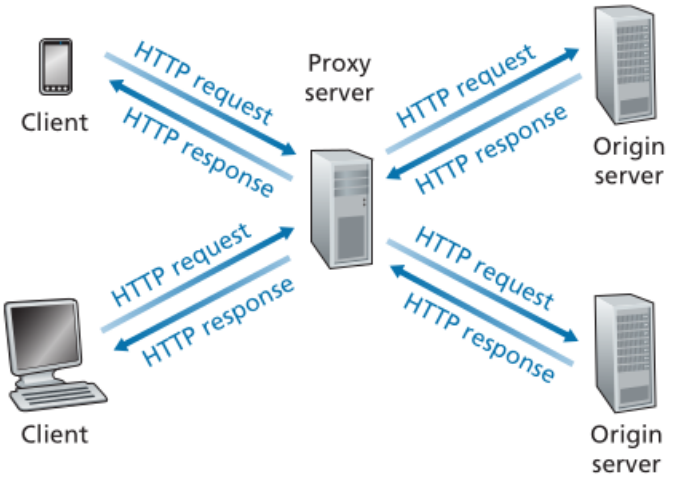


Figure 2.11 ♦ Clients requesting objects through a Web cache

1. El browser establece una conexión TCP con el Web cache y le solicita algún objeto
2. El Web caché verifica si tiene almacenada una copia del objeto solicitado. Si la tiene, la devuelve en una respuesta HTTP.
3. Si el web caché no tiene una copia almacenada, establece una conexión TCP con el servidor original y le envía la solicitud.
4. Cuando recibe la respuesta del servidor, guarda una copia del objeto y envía otra copia al cliente.

Dos ventajas importantes: reduce el tiempo de respuesta a solicitudes, pueden reducir significativamente el tráfico en los enlaces de acceso (esto reduce costos porque se ahorra tener que upgradear el ancho de banda).
Algo **negativo** que podría pasar es que el objeto cacheado no esté actualizado, para eso existe un mecanismo que permite a la caché verificar que sus objetos almacenados estén al día → **Conditional GET**. Se dice que **una solicitud HTTP es un conditional get** si: la solicitud usa el método GET y si la solicitud incluye el header **If-Modified-Since**. Lo que hace el GET condicional, es pedirle el objeto al servidor sólo si el objeto fue modificado desde la fecha indicada en **If-Modified-Since**.

Electronic Mail in the Internet

El **sistema de mails** se compone por tres elementos: **user agents**, **mail servers** y el **Simple Mail Transfer Protocol (SMTP)**.
User agents son los agentes que permiten leer, reenviar, responder, guardar y redactar mensajes; por ejemplo, Microsoft Outlook, Apple Mail, Gmail, etc.
Puerto **25** → SMTP

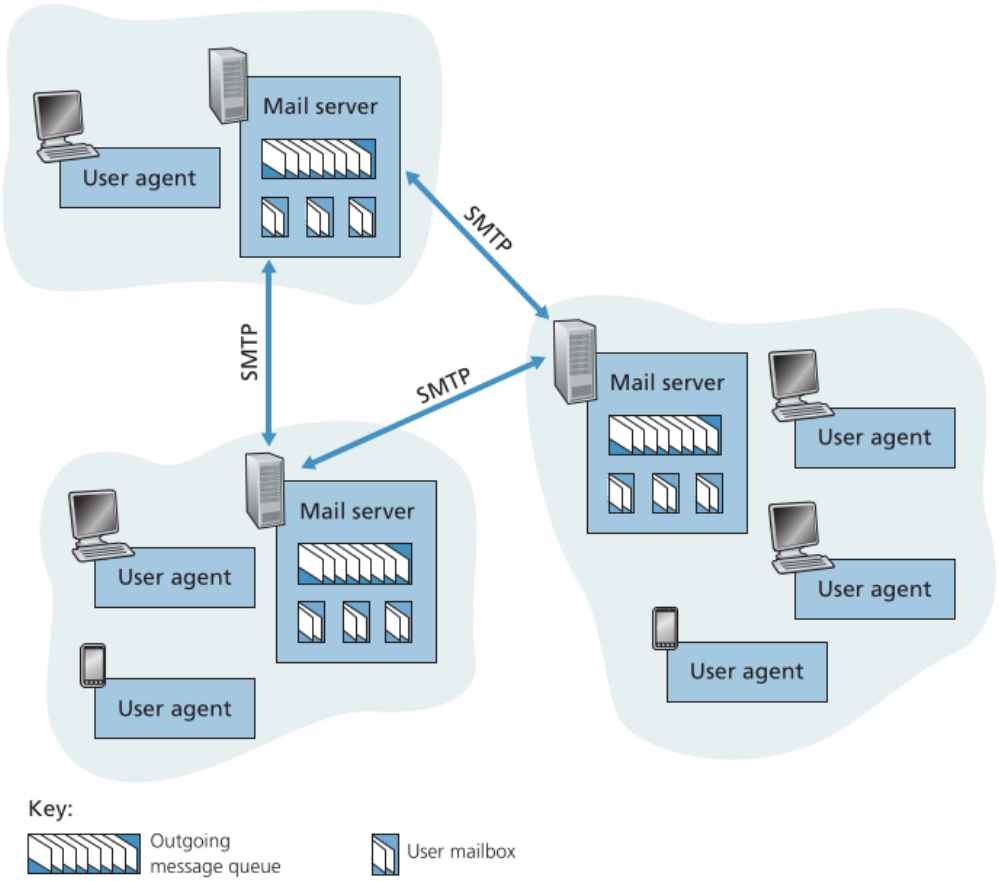


Figure 2.14 ♦ A high-level view of the Internet e-mail system

El protocolo **SMTP usa** el servicio de transferencia de datos confiable de **TCP** para transferir mails del servidor de mails del sender al servidor de mails del recipient.

El funcionamiento es asi, cuando enviás un correo, libro:
Suppose Alice wants to send Bob a simple ASCII message.

1. Alice invokes her user agent for e-mail, provides Bob's e-mail address (for example, bob@someschool.edu), composes a message, and instructs the user agent to send the message.
2. Alice's user agent sends the message to her mail server, where it is placed in a message queue.
3. The client side of SMTP, running on Alice's mail server, sees the message in the message queue. It opens a TCP connection to an SMTP server, running on Bob's mail server.
4. After some initial SMTP handshaking, the SMTP client sends Alice's message into the TCP connection.
5. At Bob's mail server, the server side of SMTP receives the message. Bob's mail server then places the message in Bob's mailbox.
6. Bob invokes his user agent to read the message at his convenience.

(ejemplo: *alice@uba.ar* ⇒ *bob@gmail.com*):

1. El ***user agent*** de Alice se conecta al **servidor SMTP de uba.ar**.
2. Este servidor busca el **servidor SMTP de gmail.com** usando DNS (registros MX).

3. Se establece una conexión TCP al puerto 25 del servidor de destino.
4. Ambos servidores intercambian comandos y respuestas de texto siguiendo la convención SMTP.
5. El mensaje se transfiere y el servidor receptor lo guarda en el buzón de Bob.
6. Cuando Bob abre Gmail, lo recupera con IMAP o POP3.

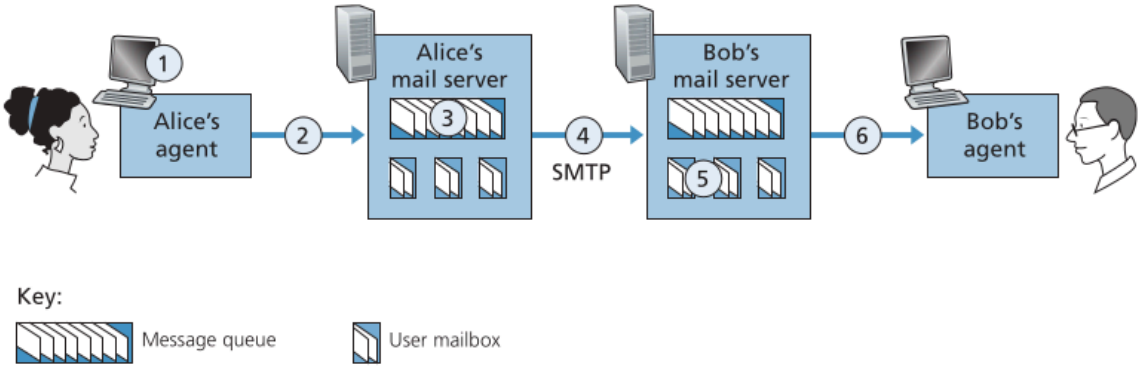


Figure 2.15 ♦ Alice sends a message to Bob

Primero, el cliente SMTP establece una conexión TCP al puerto 25 del servidor SMTP.
Si el servidor está caído, vuelve a intentar más tarde.
Una vez establecida la conexión, cliente y servidor hacen un hand-shaking donde el cliente indica el mail del sender y el del recipient.
Luego, el cliente envía el mensaje.
Se repite el proceso si el cliente necesita enviar más mensajes, sino se cierra la conexión.

Ejemplo:

```
// S: Servidor
// C: Cliente
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr ... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

Una diferencia importante entre **HTTP** y **SMTP** es que **HTTP** es un **pull protocol**, los usuarios obtienen información de un servidor web (la conexión TCP la inicia **quién quiere recibir** algo), **SMTP** es un **push protocol**, el mail server manda información a otro mail

server (la conexión TCP la inicia **quién quiere enviar** algo). Una segunda diferencia es que SMTP requiere que el cuerpo del mensaje esté en formato 7-bit ASCII. Otra diferencia es que HTTP devuelve cada objeto encapsulado en una respuesta distinta, SMTP devuelve todos los objetos en un mismo mensaje.

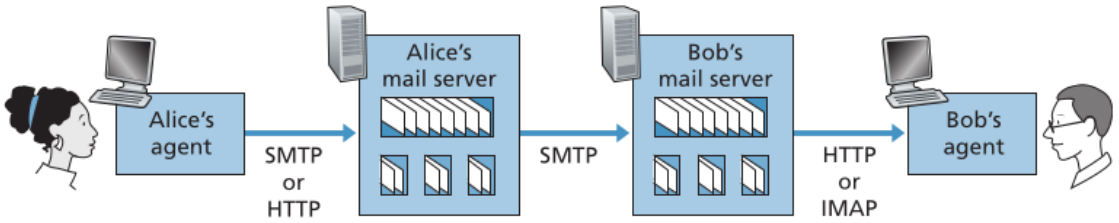


Figure 2.16 ♦ E-mail protocols and their communicating entities

Cuando se envía un email de un usuario a otro, lo que pasa es que desde el agente del enviador se empuja el mensaje al servidor del usuario vía SMTP, esto tiene la ventaja de que si el servidor del destino está caído se puede reintentar cada X minutos, ya que el servidor origen siempre está encendido. Luego, de nuevo por SMTP se manda el mensaje al servidor del usuario destino. Finalmente, el email viaja del servidor destino al agente destino mediante algún protocolo de mail access como puede ser IMAP o HTTP (Yahoo, Gmail, etc).
Hoy en día, existen dos formas comunes para que Bob recupere su correo electrónico de un servidor de correo. Si Bob usa correo web o una aplicación para smartphone (como Gmail), el agente de usuario usará HTTP para recuperarlo. En este caso, el servidor de correo de Bob debe tener una interfaz HTTP y una interfaz SMTP (para comunicarse con el servidor de correo de Alice). El método alternativo, que se suele usar con clientes de correo como Microsoft Outlook, consiste en usar el **Protocolo de Acceso a Correo de Internet (IMAP)**, definido en la RFC 3501. Tanto el protocolo HTTP como el IMAP permiten a Bob administrar carpetas almacenadas en su servidor de correo. Bob puede mover mensajes a las carpetas que crea, eliminarlos, marcarlos como importantes, etc.

DNS - The Internet's Directory Service

Hay dos maneras de identificar a un host: por su **nombre** (hostname) o por su **IP**. El **DNS (Domain Name System)** se encarga de traducir uno a otro. El DNS es una **base de datos distribuida** implementada en una jerarquía de servidores DNS y es un **protocolo de capa de aplicación** que permite a los hosts consultar la base de datos distribuida.

Un ejemplo con HTTP:

1. El browser extrae el hostname de la URL (por ejemplo, www.facebook.com) y se lo pasa al cliente DNS.
2. El cliente DNS, con el hostname, envía una consulta al servidor DNS.
3. Eventualmente, el cliente DNS recibe una respuesta con la dirección IP del hostname.
4. Una vez que el browser tiene la IP, puede abrir la conexión TCP con el servidor HTTP.

Observación: esta consulta del DNS agrega un pequeño delay.

Otros servicios que ofrece DNS:

- **Host aliasing:** un host puede tener uno o más alias si su hostname es complicado. DNS puede usarse para obtener el hostname real a partir de un alias.
- **Mail server aliasing:** de forma similar, el hostname de un correo electrónico puede ser complicado (por ejemplo que yahoo.com sea un alias de relay1.westcoast.yahoo.com).
- **Load distribution:** sitios con mucho tránsito o datos en general se distribuyen en varios servidores; tal que cada servidor corre en un host distinto y con distinta IP, o sea que una serie de IPs resultan asociadas a un hostname. La base de datos de DNS almacena este set de IPs.

Desventajas de una base de datos DNS centralizada:

- A single point of failure. Si se cae, se cae todo internet.
- Traffic volume.
- Distant centralized database.
- Maintenance.

Por eso, la base de datos se distribuye entre varios servidores.

A Distributed, Hierarchical Database

Existen 3 clases de servers DNS

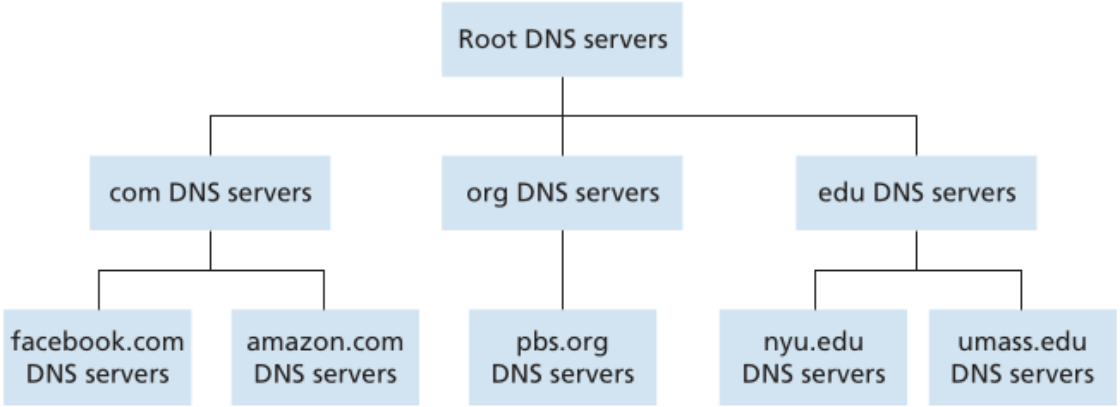


Figure 2.17 ♦ Portion of the hierarchy of DNS servers

- **Root DNS Servers:** Existen más de 1000 instancias de root servers repartidas por todo el mundo, como se muestra en la Figura 2.18. Estos root servers son copias de 13 root servers diferentes, gestionados por 12 organizaciones distintas y coordinados a través de la Autoridad de Números Asignados de Internet [IANA 2020]. La lista completa de root name servers, junto con las organizaciones que los gestionan y sus direcciones IP, se puede encontrar en [Root Servers 2020]. Los root name servers **proporcionan** las direcciones IP de los servidores **TLD**.
- **Top-level domain (TLD) servers:** Para cada dominio de nivel superior (dominios de nivel superior como com, org, net, edu y gov, y todos los dominios de nivel superior de país como uk, fr, ca y jp), existe un servidor TLD (o clúster de servidores). La empresa Verisign Global Registry Services mantiene los servidores TLD para el dominio de nivel superior com, y la empresa Educause mantiene los servidores TLD para el dominio de nivel superior edu. La infraestructura de red que soporta un TLD puede ser extensa y compleja; consulte [Osterweil 2012] para obtener una buena

descripción general de la red de Verisign. Consulte [TLD list 2020] para obtener una lista de todos los dominios de nivel superior.

- **Authoritative DNS servers:** Toda organización con hosts de acceso público (como servidores web y de correo) en Internet debe proporcionar registros DNS de acceso público que asignen los nombres de dichos hosts a direcciones IP. El servidor DNS autorizado de una organización aloja estos registros DNS. Una organización puede optar por implementar su propio servidor DNS autorizado para almacenar estos registros; como alternativa, puede pagar para que estos registros se almacenen en un servidor DNS autorizado de algún proveedor de servicios. La mayoría de las universidades y grandes empresas implementan y mantienen su propio servidor DNS autorizado, tanto primario como secundario (de respaldo).

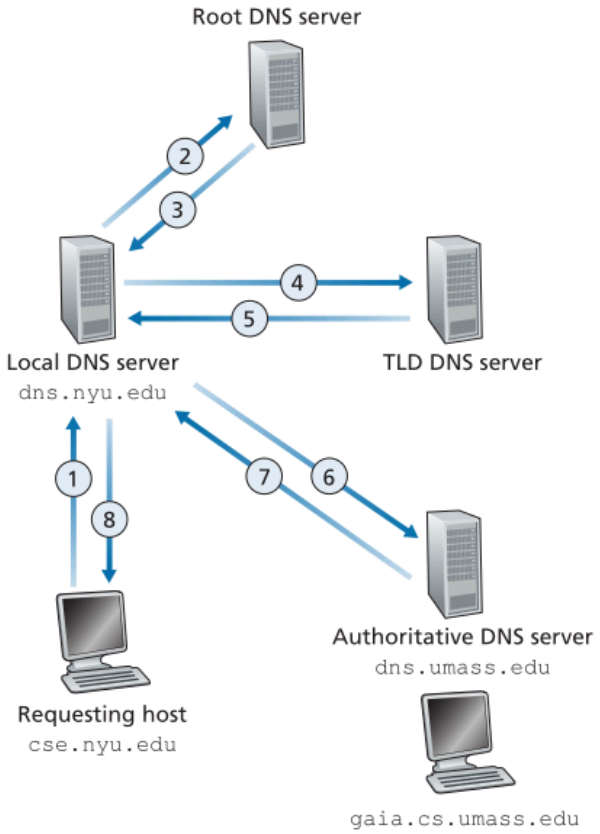


Figure 2.19 ♦ Interaction of the various DNS servers

Supongamos que el host **cse.nyu.edu** desea la **dirección IP** de **gaia.cs.umass.edu.** Supongamos también que el servidor DNS local de NYU para **cse.nyu.edu** se llama **dns.nyu.edu** y que un servidor DNS autorizado para **gaia.cs.umass.edu** se llama **dns.umass.edu.**

Como se muestra en la **Figura 2.19**:

- El host cse.nyu.edu primero envía un mensaje de consulta DNS a su servidor DNS local, dns.nyu.edu.
- El mensaje de consulta contiene el nombre de host que se va a traducir, es decir, gaia.cs.umass.edu.
- El servidor DNS local reenvía el mensaje de consulta a un servidor DNS raíz.
- Este servidor DNS raíz toma nota del sufijo edu y devuelve al servidor DNS local una lista de direcciones IP de los servidores TLD responsables de edu.

- El servidor DNS local reenvía el mensaje de consulta a uno de estos servidores TLD.
- Este toma nota del sufijo umass.edu y responde con la dirección IP del servidor DNS autorizado de la Universidad de Massachusetts, es decir, dns.umass.edu.
- Finalmente, el servidor DNS local reenvía el mensaje de consulta directamente a dns.umass.edu, que responde con la dirección IP de gaia.cs.umass.edu.

DNS usa una caché para reducir el delay y la cantidad de mensajes DNS.

Los servidores DNS almacenan resource records (RRs). Un RR es una tupla de cuatro campos: **(Name, Value, Type, TTL)**. TTL es el time to live, determina cuándo debería ser eliminado de la caché. El significado de Name y Value depende del valor de **Type**:

- Si **Type=A**: Name es un hostname y Value su IP. Ejemplo: (relay1.bar.foo.com, 145.37.93.126, A)
- Si **Type=NS:** Name es un dominio y Value el hostname de un servidor DNS de mayor jerarquía que sabe cómo obtener la IP. Ejemplo: (foo.com, dns.foo.com, NS)
- Si **Type=CNAME:** Value es el hostname canónico para el alias Name. Ejemplo: (foo.com, relay1.bar.foo.com, CNAME)
- Si **Type=MX:** Value es el nombre canónico de un mail server con el alias Name. Ejemplo: (foo.com, mail.bar.foo.com, MX)

Si un servidor DNS **tiene autoridad** para un nombre de host en particular, contendrá un registro de **tipo A** para dicho nombre. (Aunque el servidor DNS no tenga autoridad, puede contener un registro de tipo A en su caché). Si un servidor **no tiene autoridad** para un nombre de host, contendrá un registro de **tipo NS** para el dominio que lo incluye; también contendrá un **registro de tipo A que proporciona la dirección IP del servidor DNS** en el campo Valor del registro NS. Por ejemplo, supongamos que un servidor TLD edu no tiene autoridad para el host gaia.cs.umass.edu. En ese caso, este servidor contendrá un registro para un dominio que incluya el host gaia.cs.umass.edu, por ejemplo, (umass.edu, dns.umass.edu, NS). El servidor TLD edu también contendría un registro de tipo A, que asigna el servidor DNS dns.umass.edu a una dirección IP, por ejemplo, (dns.umass.edu, 128.119.40.111, A).

Formato de mensajes DNS:

- Los primeros 12 bytes componen el header. Los primeros 16 bits son el identificador de la query; este identificador se envía con la respuesta para poder matchear las respuestas recibidas con la query que se envió. Flags posibles:
  - 1 bit para indicar si el mensaje es query(0) o reply(1),
  - 1 bit para indicar en el reply si el DNS es authoritative para algún nombre consultado,
  - 1 bit para solicitar al servidor que trabaje recursivamente cuando no encuentre el record,
  - 1 bit para indicar que el servidor soporta recursión.
- La sección de preguntas contiene información sobre la consulta que se está haciendo. Incluye el nombre que se está consultando y el tipo de consulta (type a,mx, etc.).
- La sección de respuestas contiene los RRs resultantes del nombre consultado.
- La sección de autoridades contiene información sobre otros servidores con autoridad.

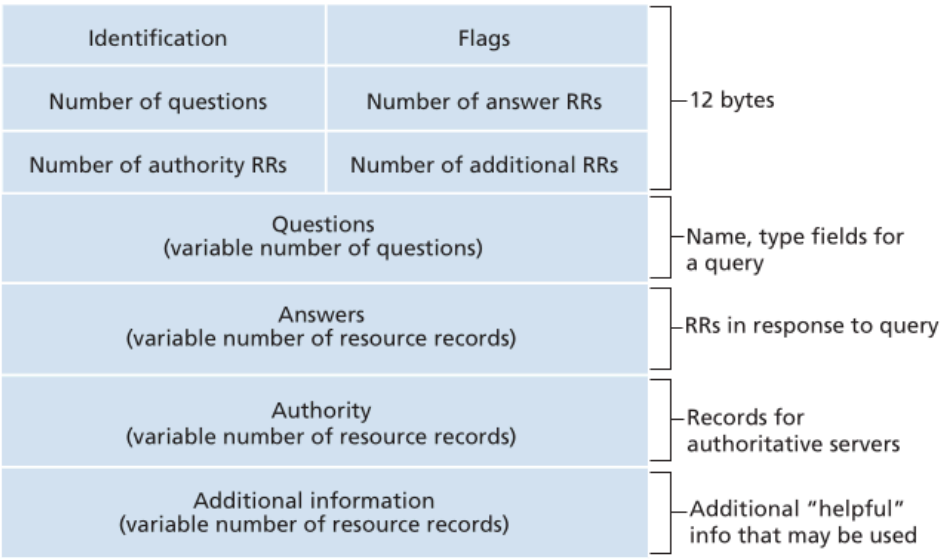


Figure 2.21 ♦ DNS message format

Peer-To-Peer File Distribution

Suponiendo que quisiéramos distribuir un archivo grande, como una nueva distribución de Linux por ejemplo, desde un servidor a varios hosts; en una arquitectura cliente-servidor, el servidor debería mandarle una copia a cada uno de los hosts, pero esto resultaría en una carga importante en el servidor y requeriría de mucho ancho de banda. En cambio, si se hace con la arquitectura **P2P**, cada peer puede redistribuir una porción del archivo a otros peers, distribuyendo la carga del servidor. Las aplicaciones con P2P pueden ser self-scaling.

BitTorrent es un ejemplo de aplicación **P2P**. Se dice que la colección de peers que participan en la distribución de un archivo es un torrent. Mientras un peer descarga porciones de algún archivo, sube también porciones del archivo a otros peers. Una vez que un usuario descargó el archivo completo, puede retirarse del torrent (selfishly) o quedarse y seguir subiendo partes del archivo para otros peers.

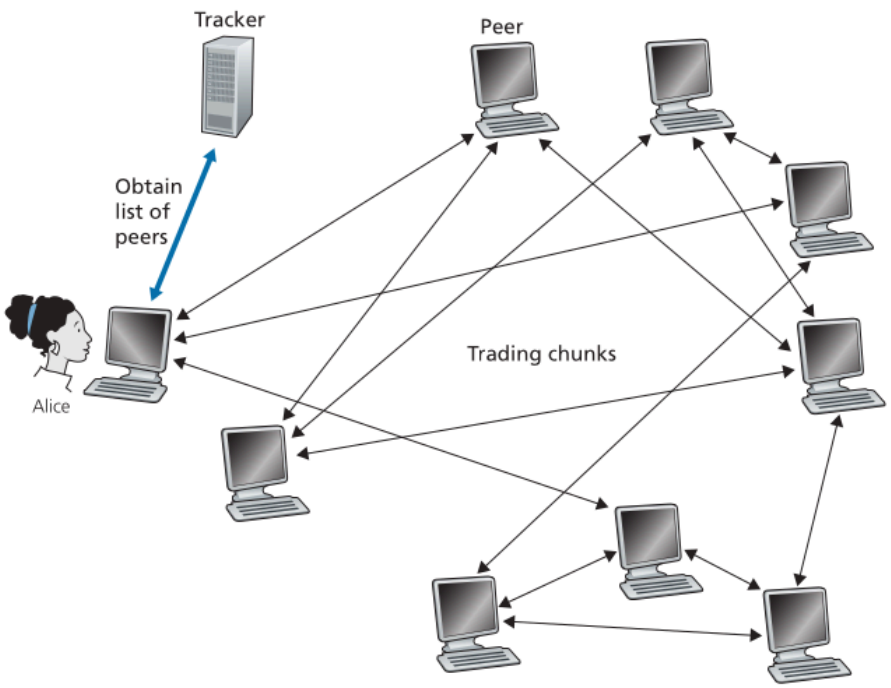


Figure 2.24 ♦ File distribution with BitTorrent

Cada torrent tiene un nodo llamado tracker. El tracker lleva registro de los peers que están participando del torrent. Cuando aparece un nuevo peer, el tracker selecciona un subset de peers y le envía al nuevo las IPs de dicho subset. Con estas IPs, el nuevo peer puede establecer una conexión TCP con cada uno. Con el tiempo habrá peers que se vayan y otros nuevos, las conexiones van fluctuando. Periódicamente, a partir de estas conexiones TCP se consulta a los otros peers vecinos qué porciones del archivo tienen; se comparan los propios con los ajenos y se piden los que no tenga.
Para decidir a qué vecinos pedirles datos se utiliza el mecanismo **rarest first**, la idea es pedir los bits menos repetidos entre los vecinos así se distribuyen equitativamente las partes.
Para decidir a qué vecinos enviarles información solicitada, se le da prioridad a aquellos vecinos que están compartiéndole datos a más velocidad pero también incluye a uno seleccionado de manera random.

En servicios que proveen **contenidos multimedia**, un **primer approach** que se podría pensar es que tengan un data center con toda la información almacenada allí y que hagan el stream desde ahí a cualquier parte del mundo. El problema con eso es que:

1. Si el cliente está muy lejos del data center, los paquetes van a tener que pasar por un montón de enlaces y routers, y eso puede terminar en delays muy feos;
2. Un video popular va a estar enviándose muchas veces sobre los mismos enlaces, esto gasta mucho **bandwidth** y es costoso;
3. Un solo data center es un **single point of failure**, si se cae, se cae todo.

Como solución, aparecen las **CDN (content distribution networks)**. Una CDN tiene múltiples servidores distribuidos por todo el mundo, tal que intenta redirigir las solicitudes de cada cliente al **CDN** que le provea la mejor experiencia.

Como algunos videos no son muy populares, algo que hacen muchos CDNs es guardar los videos en algunos clusters, si un cluster necesita un video que no tiene, entonces se lo pide a otro y se guarda una copia local al mismo tiempo que hace streaming del video.

Muchas CDNs aprovechan DNS para redirigir solicitudes. Por ejemplo: un proveedor NetCinema distribuye sus vídeos mediante KingCDN, en el sitio de NetCinema las urls tienen la forma http://video.netcinema.com/6Y7B23V. Lo que sucede:

1. El usuario ingresa a la url y el host envia la solicitud DNS para video.netcinema.com
2. El DNS local del usuario envía la solicitud al servidor de mayor jerarquía de NetCinema. Este último, en vez de devolver la IP, devuelve otro hostname del dominio de KingCDN, podría ser por ej. a1105.kingcdn.com
3. Ahora el DNS local envía una segunda query pero esta vez al servidor DNS de KingCDN y este sí devuelve la IP.

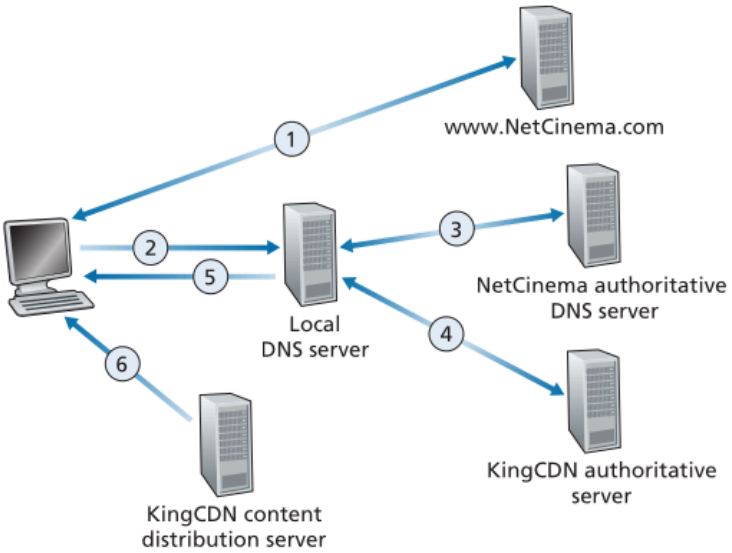


Figure 2.25 ♦ DNS redirects a user's request to a CDN server

Capítulo 3: Transport Layer

La capa de transporte es la que permite que dos programas que están en computadoras distintas pueden comunicarse entre sí. No conecta solo máquinas, sino procesos. Por ejemplo, cuando uso Chrome y entro a una página, Chrome se comunica con el servidor web gracias a esta capa.

Esta capa se apoya en la capa de red, que solo lleva paquetes entre hosts, pero no se fija que programa los usa. Entonces la capa de transporte agrega esa inteligencia.

En internet, esta capa ofrece dos tipos principales de servicio: uno confiable con TCP, y otro más simple con UDP.

Relación entre la Capa de Transporte y la Capa de Red

La capa de red está un nivel más abajo que la capa de transporte en la pila de protocolos. La capa de transporte se encarga de que los procesos se comuniquen de manera “lógica”, es decir, “yo soy un proceso y quiero enviarle algo a otro, no me interesa por donde pase” y la capa de red se encarga de la comunicación entre hosts.

En el libro se ve un ejemplo muy bueno, que habla de dos casas en diferentes ubicaciones (siendo estas los hosts) y con personas dentro (siendo estos los procesos). Desde ambas casas se envían cartas (comunican paquetes), la empresa que se encarga de enviar todos los paquetes de una casa a la otra simula la capa de red y la persona encargada de distribuir esas cartas entre las personas dentro de una casa, simula la capa de transporte. Si bien vemos que una capa tiene conexión de la otra, dependen una de la otra, pero de igual manera, una capa puede proveer un servicio que la otra no. Por ejemplo, si el que recibe las cartas en una casa, recibe solo algunas o con fallas, la capa de transporte, usando TCP, puede reclamarle al cartero (capa de red) que se la mande nuevamente, y que la persona (proceso) que debería recibirla, no se entere de este suceso y solo se notifique cuando reciba su carta correspondiente en buen estado.

Multiplexación y Demultiplexación

Dentro de los hosts se pueden estar ejecutando distintos procesos, los cuales van a tener un puerto asignado, ya sea de forma automática por la capa de transporte o nosotros asociandolos un puerto, dentro de los permitidos. Cuando se envían mensajes de un proceso en un host hacia otro/s procesos en otros hosts, queremos hacerlo de la forma más prolija y rápida posible, por ende, surge “Multiplexación”. En esta acción, la capa de transporte es responsable de juntar las cartas (los mensajes de los distintos procesos) y asignarles su tupla con (ip_origen, puerto_origen, ip_destino, puerto_destino), teniendo un conjunto de estos, cada uno con su segmento, los envía a la capa de Red. Una vez llegados al host destino, la capa de transporte es la encargada de “Demultiplexar” esto que llegó, y enviar cada carta (mensaje) a su proceso destino correcto. Notar que mandar en la tupla la ip y puerto de origen, es por si el proceso que recibe quiere responder, sabe a quien. Como vimos en la definición anterior, este es el trabajo que realizan las personas que reciben las cartas en los hogares y son las encargadas de dárselas a las personas correctas.

Cuando se envía un mensaje/carta a un proceso, se envía al socket de ese proceso, por ende, va a pasar dos cosas distintas dependiendo del protocolo que se use. Si utilizamos UDP como protocolo de la capa de transporte, puede pasar que más de un mensaje proveniente de distintos hosts o procesos, pero con el mismo host y puerto destino, vayan a parar al mismo socket del proceso destino, dado que cada socket va a estar asociado sólo al puerto destino (aunque en el encabezado siguen viajando la IP y el puerto de origen para saber quién envió el mensaje). En cambio, con TCP como protocolo de la capa de transporte, cada mensaje de distinto proceso origen, va a parar un socket distinto en el proceso destino porque cada socket está asociado por la cuadrupla (ip_origen, puerto_origen, ip_destino, puerto_destino).

Finalmente, si se utiliza HTTP no persistente en la capa de aplicación, se crea una nueva conexión TCP para cada petición, y por lo tanto un nuevo socket para cada intercambio.

Transporte sin Conexión: UDP

UDP es el protocolo que surge para solamente cumplir lo indispensable, siendo este el protocolo más simple y rápido. Solamente cumple con la Multiplexación/Demultiplexación de los mensajes. No tiene control de congestión, sistema ante fallas de paquetes dañados o extraviados, y se dice que es Transporte sin Conexión porque no realiza una conexión de tres pasos entre el proceso origen y el proceso destino, como si lo hace TCP. De esta manera, si el desarrollador de la aplicación elige UDP, entonces prácticamente es la aplicación la que se comunica directamente con IP la capa de Red.

¿Por qué existen aplicaciones en donde se prefiere UDP?
Mejor control en el nivel de aplicación sobre qué datos se envían y cuando. Con UDP, tan pronto un proceso pasa datos a UDP, UDP los empaqueta en un segmento UDP e inmediatamente entrega el segmento a la capa de Red. Por el contrario, TCP tiene un control de congestión que regula el flujo del emisor TCP de la capa de transporte cuando uno o más de los enlaces existentes entre el host origen y host destino están excesivamente congestionados. Siendo este para UDP, un posible caso de pérdida de paquetes si el enlace colapsa.
Sin establecimiento de la conexión. TCP lleva a cabo una conexión de 3 fases antes de enviar datos, por ende, esto genera latencia que UDP no tiene. De igual forma, se podría implementar todas estas restricciones dentro de la capa de aplicación por encima de UDP.
Sin información del estado de conexión. Para que TCP pueda llevar a cabo el control de congestión y el servicio fiable de transferencia de datos, necesita tener información acerca del estado de los buffers de recepción de envío, parámetros de control de congestión y de los parámetros relativos al número de secuencia y de reconocimiento. Esto es mucho para cada cliente, por eso un servidor dedicado a una aplicación, puede soportar más clientes activos cuando la aplicación se ejecuta sobre UDP.
Poca sobrecarga debida a la cabecera de los paquetes. Los segmentos TCP contienen 20 bytes en la cabecera de cada segmento, mientras que UDP solo requiere 8 bytes.

Entonces, en aplicaciones donde se priorice la velocidad y se pueda soportar un margen de pérdida de datos, se va a elegir UDP, como pueden ser los streamings en vivo, aplicaciones online, etc.

Muchas aplicaciones utilizan UDP como base y luego implementan sus propios mecanismos de confiabilidad, control o seguridad en la capa de aplicación, adaptándolos a sus necesidades.

Estructura del Segmento UDP

Un segmento UDP tiene:

- Puerto origen.
- Puerto destino.
- Longitud (Los datos + header).
- Checksum.
- Datos de la aplicación (mensaje).

No hay muchas cosas porque busca ser liviano. Solo lo justo para que llegue al proceso correcto y detectar errores básicos.

Checksum en UDP

Para calcular el checksum, se suman los datos reales que forman el paquete que se va a enviar. Esto incluye el pseudo-header tomado del encabezado IP, el encabezado UDP con el campo checksum en cero, y los datos de la aplicación. Cuando tengamos ese valor, el complemento a 1 será el checksum.

Por ejemplo, supongamos que la sumatoria de todas las palabras me da 0100101011000010, entonces el checksum va a ser: 1011010100111101 (los 0 se cambian a 1 y viceversa).

Esto lo va a hacer la capa de transporte en la etapa del emisor, y le manda correctamente el segmento a la capa de red. Cuando llega a destino, la capa de transporte del lado receptor, va a realizar el mismo cálculo de todos los ítems que incluye el segmento y va a realizar la suma con el checksum. Si da todo 1 's ⇒ está todo OK, sino hubo un error. En caso de detectar un error, el protocolo UDP simplemente lo descarta o lo entrega a la capa de aplicación pero avisando que hubo error, no hace nada más.

Estos errores pueden producirse por interferencias, congestión o problemas en enlaces o routers intermedios.

Principios de Transferencia Confiable

En esta sección se va viendo de a ejemplos, cómo van surgiendo problemas y cómo se corrigen progresivamente. Se define Finite-State Machine FSM, se usa para ver los estados de las entidades durante el envío de un paquete.

En primer lugar, un ejemplo de rdt1.0 (Reliable Data Transfer), en donde es todo color de rosas, enviamos y recibimos paquetes sin problemas y sin ningún mecanismo de detección de fallas.

En segundo lugar, un ejemplo de rdt2.0, en donde ya empiezan a haber errores y definimos criterios para verlos. La **Detección de errores** como UDP, que utiliza el checksum. **Realimentación del receptor**, va a haber una “charla” entre el emisor y el receptor, en donde por cada paquete que envíe el emisor, el receptor tendrá que enviar un mensaje de ACK “Lo recibí correctamente” o NAK “Llegó corrompido”. **Retransmisión**, en caso de que el receptor envíe NAK al emisor o no reciba respuesta en un tiempo definido (timeout), éste deberá reenviar dicho paquete.
Mientras no se reciba una respuesta ACK de parte del receptor, en donde esté todo ok, el emisor no puede empezar a enviar otro paquete. Y es por esto, que protocolos como rdt2.0 se conocen como protocolo de **Stop-And-Wait**.
Surge un nuevo problema y es como hacemos si los mensajes de ACK o NAK están corrompidos? Es decir, si el mensaje que recibe el receptor de “Lo recibí” dice “awdawd”, el emisor no sabe si el paquete fue recibido correctamente o no, lo que puede generar retransmisiones innecesarias y duplicados.

Para resolver esto surge rdt2.1, que agrega números de secuencia a los paquetes. Cada paquete se identifica con un número, por ejemplo 0 o 1. De esta forma, si un paquete se retransmite por error, el receptor puede detectar que es un duplicado y descartarlo sin volver a entregarlo a la aplicación.

Luego aparece rdt2.2, que simplifica el protocolo eliminando los mensajes NAK. En lugar de usar ACK y NAK, solo se utilizan ACK con números de secuencia. El receptor envía un ACK indicando el último paquete recibido correctamente. Si el emisor no recibe el ACK esperado, asume que hubo un error y retransmite.

Luego aparece rdt3.0, en donde el emisor instala un timer para cada paquete que se envía, en donde se busca solucionar la pérdida de un paquete o ACK. Puede pasar que el envio de ambos este retrasado por X motivo (mucho trafico en la red, calculo inesperado, etc) y que el emisor mande nuevamente el paquete cuando en realidad el que envio antes, solamente esta retrasado. Aparecen los mensajes duplicados, pero con los números de secuencia, es fácil descartarlos por parte del receptor.

Protocolos Confiables con Pipeline

Hasta rdt3.0 fue todo muy lindo y fácil de entender e implementar. Pero este mecanismo en la vida real no es eficiente, imaginate pagar una fortuna por un enlace con capacidad de 1 gigabit para tener una tasa de transferencia mucho menor, ni siquiera un cuarto, se desperdicia mucho ancho de banda. El hecho de tener que esperar un ACK por cada paquete que enviamos es sin dudas un tiempo de ocio muy grande.

Por esto, surge la idea del pipeline. En lugar de enviar un solo paquete y esperar, el emisor puede enviar varios paquetes consecutivos sin esperar inmediatamente sus ACK. De esta forma, se mantienen varios paquetes “en vuelo” al mismo tiempo, aprovechando mejor el enlace.

Para esto se utiliza una “**ventana**”, que determina cuántos paquetes pueden enviarse sin haber sido confirmados. Mientras haya espacio en la ventana, el emisor puede seguir enviando.
Cada paquete sigue teniendo su número de secuencia único, que permite identificarlo y detectar pérdidas o duplicados. Los ACK sirven para confirmar la recepción correcta de los paquetes.

El tamaño de la ventana, el rango necesario de los números de secuencia y los requisitos del buffer va a depender del protocolo que se emplee para su recuperación frente a errores de transferencia. Veremos dos: **Go-Back-N** y **Selective-Repeat**.

Go-Back-N

En este protocolo vamos a enviar una cantidad “N” de paquetes sin esperar ACK. Cada paquete lo vamos a identificar con su número de secuencia único, en donde, vamos a enviarlos de forma ordenada. Tendremos una “ventana” de largo N que será la cantidad de paquetes que enviemos sin recibir dichos ACK’s.

No vamos a recibir un ACK por cada paquete que enviemos, sino, será del último paquete dentro de la ventana N que tengamos, puede pasar que tengamos menos paquetes que el largo de nuestra ventana N.

A este protocolo lo podemos pensar como una cola de paquetes ordenados por números de secuencia en la que el emisor tendrá que mantener actualizados índices para “paquete base” de la cola donde empieza mi ventana actualmente (paquete más antiguo enviado que no tengo ACK) y para “nextseqnum” que será el paquete próximo a enviar dentro de mi ventana N.

A medida que la capa de transporte recibe de la capa superior un paquete para enviar, se fija si ya completó su ventana y en caso de que no, envía, sino almacena para cuando se libere.

El receptor a medida que recibe paquetes los manda a la capa superior, entonces cuando se recibe algún paquete desordenado en relación al número de secuencia, por ejemplo, recibe paquetes 1, 2, 3, 5, 4, el receptor manda un ACK con el número de secuencia del último paquete ordenado que recibió (en este caso 3) y los demás los descarta. El emisor reenvía desde ese paquete en adelante nuevamente (reenvía 4 y 5). Se reenvian aunque hayan llegado bien dichos paquetes.

El emisor va a setear un timer por el paquete más antiguo de la ventana enviado y lo resetea cuando recibe el ACK que contempla ese paquete antiguo de la ventana. Si salta timeout antes de recibir ese ACK, reenvia la ventana completa.

Selective-Repeat

Dado que **Go-Back-N** no es tan eficiente por el hecho de que si tenemos una ventana de 1000 palabras, las enviamos todas y el receptor nos manda que de la 3er

palabra en adelante recibió desordenados, tenemos que reenviar nuevamente casi 1000 palabras. Surge este protocolo en el que, como su nombre indica, reenviamos aquello que no se recibió correctamente.

Ambos, emisor y receptor, van a mantener una “ventana” de N paquetes enviados y recibidos, el receptor va a aceptar paquetes fuera de orden. Puede que no coincidan una con la otra, pero esto se va a entender ahora.

Ahora, va a haber un ACK por cada paquete enviado, pero sigue la posibilidad de enviar, en el margen de la ventana N, paquetes sin recibir ACK. El emisor va a tener una “cola” con los paquetes ordenados con su número de secuencia único. El emisor va a enviar la cantidad de paquetes que le permita la ventana y va a ir avanzando la ventana, es decir, corriendo a la derecha la posibilidad de seguir enviando, a medida que reciba el ACK del paquete más antiguo.

Vamos con un ejemplo para entender:
Supongamos que tenemos una ventana con N = 3.
El emisor envía los paquetes 0, 1, 2, 3.
El receptor sólo recibe el paquete 0, 1 y 3, entonces envía el paquete 0 y 1 a la capa superior, almacena en el buffer el paquete 3 y envía los ACKs de los paquetes que recibió, osea, 0, 1 y 3.
El emisor recibe el ACK del paquete 0, avanza su ventana y envía el paquete 4, pasa lo mismo con el paquete 1 y envía el paquete 5.
El receptor recibe correctamente los paquetes 4 y 5, los almacena en el buffer y envía los ACKs para 4 y 5.
El emisor no puede avanzar en la ventana dado que todavía no recibe el ACK del paquete extremo que es el número 2.
Salta timeout por ese paquete y lo reenvía.
El receptor lo recibe, manda el ACK del paquete 2.
El receptor ordena los paquetes almacenados en el buffer y los envía a la capa superior.
El emisor recibe el ACK del paquete 2 y puede correr su ventana al paquete número 5.

Transporte Orientado a Conexión: TCP

Breve introducción, destaca que el protocolo TCP confía mucho en los principios definidos anteriormente y está definido en los documentos RFC 793, RFC 1122, RFC 1323, RFC 2018 y RFC2581.

La Conexión TCP

Decimos que TCP es de conexión porque antes de enviarse datos entre un emisor y receptor, se establece una conexión entre ambos.

Se definen buffers, variables y sockets únicos por cada conexión entre un proceso A en un Host 1 y un proceso B en un Host 2. En estos casos se dice que la conexión es punto a punto, dado que es de suma complejidad realizar una conexión de uno a muchos. En TCP se dice que 2 es compañía y 3 ya es multitud.

Estos buffers y variables que se definen, son solamente en la capa de transporte, del emisor y el receptor. La capa de Red no tiene idea de que está pasando una conexión TCP, estos buffers y variables no se definen en los routers o switches.

Además, una conexión TCP es full-duplex, lo que significa que ambos extremos pueden enviar y recibir datos al mismo tiempo.

La conexión en TCP consta de 3 pasos, el emisor que será el **cliente** porque inicia la conversación, manda un mensaje especial de TCP hacia el proceso en otro host con el que se quiere comunicar. Este proceso que recibe ese mensaje especial es el receptor que tendrá el rol de **servidor** por recibir el primer mensaje especial. El receptor le enviará un mensaje especial TCP como respuesta al emisor. El emisor recibe ese mensaje y le manda otro mensaje especial de TCP al receptor, siendo este el último para establecer la conexión.

En estos mensajes de inicio, se comparten ambos procesos, su número de secuencia. Este número se genera aleatoriamente y sirve para detectar fraude entre conexiones, si la conexión es muy vieja, etc. etc.

Mensajes en este orden: SYN ⇒ SYN-ACK ⇒ ACK

Al ser una conversación de 3 mensajes para iniciar la conexión, se dice que TCP consta de un **three-way handshake** o **conexión en 3 fases**. Esta conexión en 3 fases se puede entender, por ejemplo, **cliente** ⇒ Hola!, **servidor** ⇒ Hola!, **cliente** ⇒ Empiezo a comunicarme.

Existe **MSS** Maximum Segment Size, durante el establecimiento de la conexión también se negocia el **MSS**, que es el tamaño máximo del segmento. El **MSS** se calcula a partir del **MTU** del enlace, restando el tamaño de los headers IP y TCP.

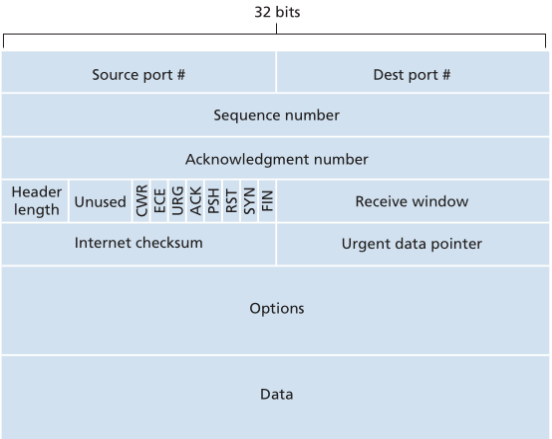
Este protocolo consta de buffers, uno en el proceso que envía y otro en el proceso que recibe porque no se mandan los paquetes al instante que la capa superior lo pida. Después vemos…

Estructura del Segmento TCP

La estructura del header de TCP va a ser más grande que la de UDP, el header de TCP consta de 20 bytes y el de UDP 8 bytes. TCP al igual que UDP va a tener: **Número de puerto origen y destino**, para multiplexar y demultiplexar, **campo de checksum** y también contiene:

- **campo Número de secuencia** de 32 bits: Número inicial del flujo de bytes que tiene el segmento. TCP percibe los datos como un flujo de bytes no estructurado pero ordenado, estos números de secuencia hacen referencia al flujo de bytes transmitidos y no al flujo de segmentos transmitidos. Cada proceso al hacer el handshake se manda su número de secuencia elegido aleatoriamente, este va a ser su número de secuencia inicial respectivamente.
- **campo Número de reconocimiento** de 32 bits: Es número siguiente al último byte que recibí, es decir, espero el flujo de este número de bytes en adelante.

- **Ventana de recepción** de 16 bits: Control de flujo.
- **campo Longitud de header** de 4 bits: Longitud del header TCP en palabras de 32 bits, puede ocupar más dependiendo del **campo Opciones** pero en general, 20 bytes.
- **campo Opciones** es opcional y longitud variable.
- **campo Indicador** tiene 6 bits: bit **ACK** se utiliza para avisar que el segmento contiene un **número de reconocimiento** válido. Los bits **RST**, **SYN** y **FIN** se utilizan para establecer y cerrar conexiones. Los bits **CWR** y **ECE** son para control de congestión. La activación del bit **PSH** indica que el receptor deberá pasar los datos a la capa superior de manera inmediata. El bit **URG** se utiliza para indicar que hay datos en este segmento que la entidad de la capa superior del lado emisor, ha marcado como "urgentes". La posición de este último byte de datos urgentes se indica mediante el **campo Urgent data pointer** de 16 bits.



Supongamos una transferencia de datos entre Host A (emisor) y Host B (receptor):
Número de secuencia: Si Host B recibe un segmento con bytes de 0 a 535 y otro segmento con bytes de 900 a 1000. Por alguna razón, se perdieron esos bytes de 536 a 899, entonces el Host B en el siguiente segmento que le mande, contendrá el número 536 en el campo de **Número de reconocimiento**, para decirle al Host A que le envíe nuevamente ese segmento. Pueden pasar distintas cosas dependiendo como se defina el protocolo pero generalmente, el Host B almacena los bytes de 900 a 1000 y luego ordena cuando le lleguen los bytes que faltan.
Número de reconocimiento: Yo como Host B, al recibir correctamente una tanda de bytes, del Host A, espero el número de secuencia siguiente de bytes, teniendo en cuenta el número de secuencia inicial que definió el Host A y no yo Host B. Y viceversa.

Estimación de RTT y Timeout

TCP maneja un timeout como en el modelo rdt para justamente, responder ante la falla de un envío. TCP cuenta con un Timeout adaptativo, primero calcula un **RTTMuestra** que viene a ser el tiempo que tarda cada segmento en ir al destino y recibir el ACK de dicho segmento, a medida que se envían paquetes, se va calculando un **RTTEstimado** para la conexión, en donde, es el promedio que vienen manejando los segmentos y la fórmula es un promedio ponderado:
RTTEstimado = 0,875 x RTTEstimado + 0,125 x RTTMuestra

Se asigna un mayor peso al valor recientemente estimado antes que a los más antiguos, ya que las muestras más nuevas reflejan mejor la congestión que existe actualmente en la red.

También se define la variación entre estos, como:
RTTDesv = (1 - 0,25) x RTTDesv + 0,25 x (| RTTEstimado - RTTMuestra |)

El intervalo de **Timeout** se define de la siguiente manera:
TimeoutInterval = RTTEstimado + 4 x RTTDesv

Al iniciar, se define **TimeoutInterval** igual a 1 segundo y se duplica para el segmento que salta **timeout**, para asegurarnos que no hayan retransmisiones innecesarias, no hay problema con este porque tan pronto se recibe un segmento y se actualiza el valor de **RTTEstimado**, el valor de **TimeoutInterval** se calcula denuevo con la fórmula anterior. Cada vez que TCP obtiene una nueva muestra de RTT válida, actualiza el RTT estimado, la desviación y recalcula el TimeoutInterval.

Transferencia Confiable en TCP

Ya sabemos que la capa de Red no es confiable, solo que hace su mayor esfuerzo por mandar los datagramas de un host a otro. En este trayecto los datagramas pueden sufrir reordenamientos, corromperse o extraviarse, en consecuencia, se perjudican los segmentos que llevan dentro. Para esto, TCP busca garantizar fiabilidad sin importar que dependa de la capa de Red.

TCP utiliza un timer para el segmento más viejo sin ACK de segmentos enviados, un buffer receptor puede recibir la tanda desordenada, es decir, bytes del 0 al 535 y otro segmento de 900 a 1000. No recibió el segmento con la tanda de bytes de 536 a 899, entonces le envía un ACK al emisor con el número de secuencia 536. El emisor cuando recibe este ACK, no reenvia el segmento pedido de inmediato, espera a recibir 3 ACKs duplicados o que salte el timeout (lo que pase primero), esto es **fast retransmit**.

Si se manda dos segmentos, se pierde el primer ACK pero se recibe el segundo ACK dentro del timer, el emisor entiende que el receptor recibió los dos segmentos y no envía nuevamente el primer segmento.

Si se manda dos segmentos, el receptor envía los dos ACKs correspondientes, pero estos tardan en llegar saltando el timeout, el emisor reenvía el primer paquete, recibe ambos ACKs que tardaron en llegar y acto siguiente recibe otro ACK del segundo segmento (**ACK acumulativo**) indicando que el receptor ya tiene en su totalidad ambos segmentos.

Control de Flujo

En una conexión TCP, donde hay dos procesos en distintos host comunicándose, supongamos Host A como emisor y Host B como receptor, hay un buffer en cada uno, buffer emisor y buffer receptor. A medida que el Host B recibe segmentos, van directo al buffer de recepción y no los procesa de manera inmediata necesariamente, puede que el Host B

procese en mayor tiempo del que llegan y por ende, llenarse el buffer ⇒ se descartan paquetes.

Para que esto no suceda, se define el Control de Flujo, en donde el Host A va a tener dos variables que debe tener siempre actualizadas, **UltimoByteProcesado** que hace referencia al último byte que proceso el Host B y **UltimoByteEntregado** que hace referencia al último byte que se entregó al Host B. Host B tiene el valor total de capacidad de su buffer que será **BufferRecepcion**. La resta entre **UltimoByteEntregado - UltimoByteProcesado** debe ser menor a **BufferRecepcion**.

Del lado del Host A, también se tendrá una variable **VentanaRecepcion** que será la capacidad que tiene el buffer de Host B actualmente. **VentanaRecepcion** sale de la cuenta: **VentanaRecepcion = BufferRecepcion - (UltimoByteEntregado - UltimoByteProcesado)**

Si en algún momento, este buffer de recepción se llena, el Host A se queda pausado sin poder mandar paquetes hasta que el Host B procese alguno, puede que el Host B no mande ningun paquete de confirmación a Host A, dado que no tiene nada que comunicar, entonces el Host A, una vez que se llene el buffer de recepción, seguirá mandando paquetes de un byte esperando que el Host B le responda con un valor de **VentanaRecepcion** distinto de 0.

Administración de Conexiones TCP

Hablamos de que la conexión en TCP se hace a través de un Handshake de 3 fases. Vamos a explicarlo con un ejemplo:
Dado Host A como emisor cliente y Host B como receptor servidor.
El Host A quiere iniciar conexión con un proceso ubicado en el Host B, por ende, manda un mensaje especial de TCP con el flag de SYN prendido y su número de secuencia inicial aleatorio.
El Host B recibe dicho mensaje de TCP y está de acuerdo con iniciar la conexión, entonces le manda un mensaje especial de TCP con el flag de SYN prendido (sería el SYNACK) y su número de secuencia inicial aleatorio. Además ya establece el buffer y variables para dicha conexión.
El Host A lo recibe y le manda nuevamente otro mensaje de TCP como “Conexión establecida” pero con el flag de SYN apagado, el número de secuencia que espera el servidor y el número de ACK que espera del servidor. Este mensaje es para terminar de decir “Iniciamos correctamente la conexión” y avisarle al servidor que captó correctamente su número de secuencia inicial. También en este mensaje, el cliente ya puede mandar información dentro de los límites de la sección data.

Si se quiere dar por finalizada la conexión, cualquiera de los dos puede hacerlo. Vamos con un ejemplo en el que el Host B quiere finalizar.
Host B manda un mensaje TCP especial a Host A, en donde tiene el flag FIN prendido.
Host A manda un ACK a Host B confirmando que recibió el mensaje de FIN.
Host A manda un mensaje TCP especial a Host B, en donde tiene el flag FIN prendido.

Host B manda un ACK a Host A confirmando que recibió el mensaje de FIN y pone un timer de, por ejemplo, 30 segundos por si el ACK a Host A se pierde.
Pasado el timer, Host A y B liberan todas las variables y buffers utilizados para la conexión TCP.

Principios de Control de Congestión

Hablamos del control de congestión exclusivamente en la capa de transporte, pero ahora buscamos entender cómo sucede la congestión de la red en un ámbito más general, en la capa inferior puede ocurrir un desborde de routers, corrupción de paquetes, pérdida, etc. que ya no dependen de la capa de transporte. Sin embargo, se puede tomar precauciones que ayuden a controlarlo desde la capa de transporte.

Control de Congestión en TCP

Arrancamos viendo 3 escenarios de congestión.

Escenario 1:
Dos Hosts A y B comparten un router con enlace de transmisión con capacidad R, para una velocidad de transmisión comprendida entre 0 y R/2, la tasa de transferencia del receptor es igual a la velocidad de transmisión. No hay control de flujo, el buffer del router es ilimitado.
Trabajar al límite no es siempre bueno, si alguno de los hosts empieza a mandar por encima de R/2, se empieza formar una cola de manera exponencial.
La capacidad de R es la velocidad con la que transmite paquetes, no se transmiten paquetes en simultáneo, por ende, al enviar paquetes con una velocidad cercana de la capacidad del enlace, el router empieza a acumular paquetes, y por ende, se forma una cola cada vez mayor.

Escenario 2:
Sumado al escenario anterior se le suma que los hosts tienen mecanismos de entrega fiable y que el router tiene un buffer limitado, por ende, al generarse colas de paquetes, puede crear falsas retransmisiones por parte de la capa de transporte que transcurrido un tiempo, piensa que el paquete se perdió. Con esto lo pone en un escenario peor, donde se tiene que realizar retransmisiones de paquetes que fueron transmitidos correctamente, desaprovechando el ancho de banda.

Escenario 3:
Otro gráfico en el que participan más hosts y hay varios routers con buffers finitos de por medio, se detalla el peor caso en el que un paquete realiza una ruta larga, en la que pasa por routers y enlaces, al llegar al último router, este lo descarta porque está lleno, desperdiciando todo el trabajo que hizo para llegar hasta ese punto. Se genera un círculo vicioso donde la mayor parte del tráfico son retransmisiones, haciendo que el throughput util tienda a cero.

Métodos para controlar la congestión

Dos métodos más comunes de control de congestión que se utilizan en la práctica:

Control de congestión terminal a terminal: La capa de red no proporciona ninguna información sobre la congestión en la red a los sistemas terminales, capa de transporte, por ende tienen que basarse únicamente en las fallas que ellos detectan. Mediante su

mecanismo de timeout o 3 ACK duplicados, es que deducen que hay congestión en la red y TCP en consecuencia, reduce su ventana de transmisiones.

Control de congestión asistido por la red: Como lo dice el nombre, la capa de red, el router es capaz de avisar a la capa superior (ya sea al emisor o receptor) que está congestionada con el simple hecho de mandar un bit, esto es un mecanismo que se empezó a implementar en IBM y hoy en dia los protocolos UDP y TCP pueden incorporar. También los routers son capaces de decirle al host emisor, su velocidad de transmisión de entrada y de salida. Hay dos formas en la que el router avisa que está congestionado, manda un paquete al emisor bloqueante que dice "¡Hay congestión!" o espera a que reciba un paquete en donde activa un flag de congestión, este último puede tener un retardo de ida y vuelta de un paquete normal.

Control de congestión TCP

Ya tenemos las variables definidas para el control de flujo que son **VentanaRecepcion, BufferRecepcion, UltimoByteEntregado, UltimoByteProcesado.** Ahora vamos a sumar **VentanaCongestion** que como TCP no tiene un control de congestión asistido por la red, sino que es terminal a terminal, esta variable va a variar dependiendo de las siguientes situaciones:

- Si el Emisor manda paquetes y recibe correctamente los ACK correspondientes ⇒ No hay congestión en la red.
- Si el Emisor manda paquetes pero salta el timeout o recibe triple ACK duplicado ⇒ Hay congestión en la red.

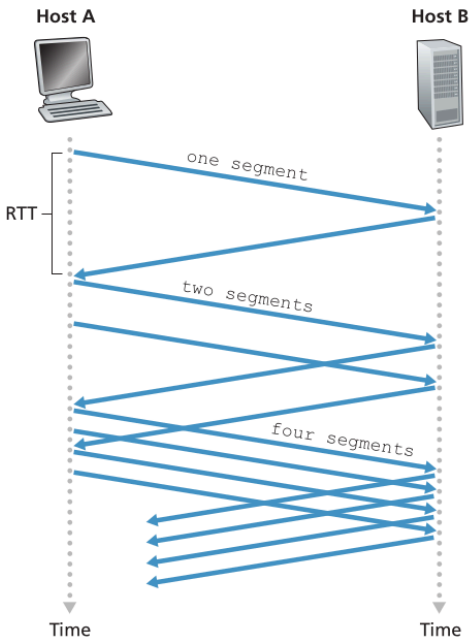
El valor de **VentanaCongestion** va a variar todo el tiempo en cuestión de los paquetes enviados dependiendo de las acciones que sucedan explicadas anteriormente. Si detecta que no hay congestión, entonces va a empezar a mandar más rápido. Si detecta congestión en la red, entonces al retransmitir, va a bajar la velocidad de transmisión. Todo esto es sin tener en cuenta el control de flujo! En ese caso la velocidad de transmisión es **min**(**VentanaCongestion, VentanaRecepcion**).

Esto es solo un pantallazo, el algoritmo de control de congestión consta de 3 componentes principales: **slow start, congestion avoidance** y **fast recovery.**

Slow Start

Cuando se inicia una conexión TCP, el valor de **VentanaCongestion** (**cwnd**) normalmente se inicializa con un valor pequeño de 1 MSS (Maximum Segmentation Size, máximo valor de segmento). Con esto una velocidad de transmisión inicial es igual a MSS / RTT. Por ejemplo, MSS = 500 bytes y RTT = 200 milisegundos, la velocidad de transmisión inicial será aproximadamente de 20 kbps.

Como puede ser que no se está aprovechando el ancho de banda, a medida que el emisor recibe el ACK de un paquete enviado, incrementa en 1 MSS la **cwnd.** En consecuencia, por cada tanda de paquetes transmitidos, se duplica el valor de la ventana de congestión.



Este crecimiento exponencial se detiene cuando se produce una pérdida de paquete (hay congestión), entonces pasa esto:
Si la pérdida es por timeout:
El valor de **cwnd** se establece nuevamente en 1 MSS y vuelve a **Slow start**.
Si recibe 3 ACK duplicados:
⇒ **fast recovery**

¿Tiene sentido seguir en slow start hasta que haya pérdida? No, por eso al tener la primera pérdida, con el valor de **cwnd** cuando se produjo esta pérdida, se define el valor del umbral **ssthresh** como **cwnd / 2**. Ahora, el algoritmo indica que va a estar en **slow start** hasta que:

- Se produce una pérdida: Realiza una retransmisión rápida y entra al estado de **fast recovery**.
- **cwnd >= ssthresh:** Sale del estado **slow start** y pasa al estado **congestion avoidance**.

Congestion Avoidance

En vez de tener un crecimiento exponencial, crece linealmente, se suma 1 MSS por cada RTT (envuelve una tanda de segmentos enviados). Si hay pérdida, pasa lo siguiente dependiendo el caso:

- Timeout:
  ssthresh = cwnd / 2
  cwnd = 1
  ⇒ vuelve a **slow start**
- 3 ACK duplicados:
  ⇒ entra en **fast recovery**

En slow start, la ventana aumenta en 1 MSS por cada ACK recibido, por lo que en cada RTT se duplica aproximadamente. En cambio, en congestión avoidance, la ventana aumenta mucho más lento, linealmente: en total crece solo 1 MSS por RTT, independientemente de la cantidad de ACK recibidos.

Fast Recovery

Existen dos protocolos dependiendo cual implementa TCP, existe **TCP Reno** que lo que hace es:

ssthresh = cwnd / 2
cwnd = ssthresh + 3 (por los 3 ACK duplicados)
⇒ sigue en **Congestion Avoidance**

Y existe **TCP Tahoe** que hace:

ssthresh = cwnd / 2
cwnd = 1
⇒ vuelve a **slow start**

Capitulo 4, The Network Layer: Data Plane

Data Plane: Se encarga de reenviar paquetes, es decir, decidir por que interfaz de salida mandar cada paquete, usando la tabla de forwarding. No calcula rutas. No decide caminos. Solo ejecuta lo que el **Control Plane** le dijo.

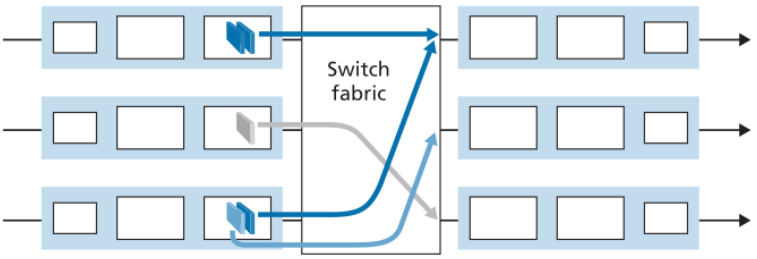
Dirección IP: Una dirección IPv4 tiene 32 bits:
192.168.1.25
se divide en [Parte de Red | Parte de Host]
Esto se define con la máscara.

por ejemplo:
IP: 192.168.1.25
Máscara:255.255.255.0 → /24
⇒ **Red: 192.168.1**
Host: 25
Todos los que tengan 192.168.1.x están en la misma red

Arquitectura router

Input port (interfase de entrada): Un paquete entra al router, viene de pasar por la capa física que recibe los bits y arma la trama, pasa por capa de enlace en donde saca el encabezado Ethernet y finalmente queda como datagrama IP en el router, capa de Red. Ahora dentro del router lee la IP destino, mira en su **Forwarding Table** (actualizada por Control Plane), esta tabla se utiliza con LPM (Longest Prefix Match) y de todas las IPs que marca esta tabla en que puerto salir, matchea con la que más dígitos matchee.

Switch fabric (conmutación): Es quien conecta los puertos de entrada con los puertos de salida dentro del router. Existen 3 tipos:
Switching via memory: El paquete pasa por la memoria central. Es lento y cuello de botella, hoy no se usa.
Switching via Bus: Todos comparten un bus interno. Más rápido pero si muchos quieren usarlo se satura.
Switching via Crossbar (moderno): Matiz de conexiones paralelas, muchos paquetes al mismo tiempo como se ve en la imagen, a diferencia de los otros dos que solo puede ir uno a la vez, con esto se elimina el cuello de botella y permite alto throughput. Paquetes que van a distintos puertos de salida pueden hacerlo al mismo tiempo pero si van al mismo puerto de salida, uno debe esperar. Es mecanismo para decidir quién va primero depende la implementación pero puede ser por prioridades, round robin, etc.



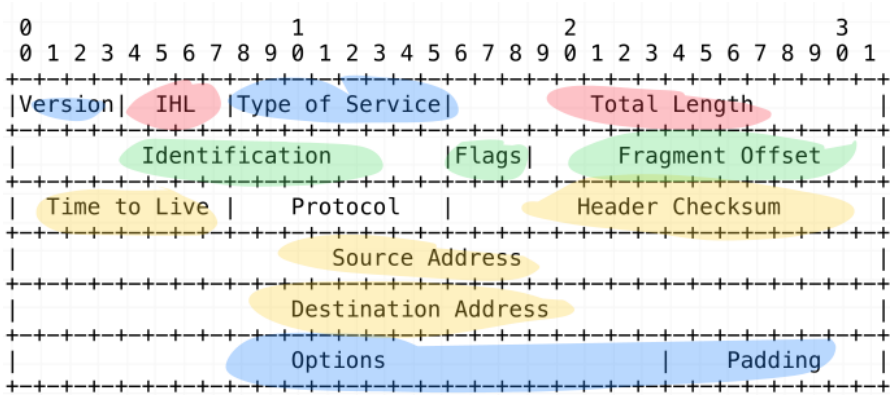
Para las primeras veces del paquete es routing en PC tarda O(n) y luego usa hash para las siguientes veces que tarda O(1).

Output port: Por aca solo sale, si hay paquetes, va a la cola y si esta está llena, pérdida (congestión en el router), se encapsula nuevamente según la tecnología del enlace y se transmite.
Por lo general, es acá donde se produce la congestión, en donde, varios paquetes con input distinto quieren salir por el mismo puerto de output.

Gestión de los paquetes encolados: Esto sucede en los puertos de Input y output, el scheduler es quien decide cómo avanza esa cola, por prioridad, round robin, etc.

RED: Random Early Detection ⇒ Para no esperar a que se llene el buffer, va descartando paquetes de cada flujo aleatoriamente.

Datagramas IP



• Versión: Acá se ve si es IPv4 (con 4) o IPv6 (con 6).
• direcciones: Origen IP, Destino IP
• protocolo: TCP (6), UDP (17), ICMP (1)
• verificación (checksum): Verifica errores del header, no datos. Si falla ⇒ descarta.
• robustez: TTL, cantidad de saltos restantes. Si llega a 0 ⇒ descarta. Evita loops infinitos.
• tamaño del encabezado (IHL): Indica longitud del header. Normalmente 20 bytes pero con opciones más.
• fragmentación (ID, flags, o -set): Sirve para cuando hubo fragmentación del paquete por ser mayor al tamaño MTU. Identification: ID común a todos. Flags: DF no fragmentar o MF más fragmentos. Offset: posición, para rearmar el paquete.
• Datos: payload

Se pidió: “Capturar la salida de un comando `ping' a cualquier host durante al menos 2 horas, y graficar su distribución en escala lineal y luego en escala log-log. Entregar un pequeño informe en PDF por el campus.”
Mostraba que la latencia en Internet no es constante, sino que presenta variabilidad y eventos extremos, evidenciados en una distribución con cola pesada en escala log-log, asociados a congestión y buffering.
Log-log: ambos ejes están en escala logarítmica y se pueden apreciar gran rango de valores y destacar colas pesadas o comportamientos de tipo ley de potencia.

Enrutamiento sin clases: Subredes
Hoy se usa **CIDR** (Classless Inter-Domain Routing, Enrutamiento sin clases) en donde vos elegís cuantos bits son de red IP/prefijo, optimizando el uso de direcciones y el enrutamiento.

Antes era por clases:

Clase	Prefijo	Hosts
A	/8	millones
B	/16	miles
C	/24	254

DHCP (Dynamic Host Configuration Protocol): Es un protocolo cliente-servidor que provee parámetros de configuración automática a los hosts. Utiliza asignación dinámica mediante leases y funciona mediante el intercambio DISCOVER, OFFER, REQUEST y ACK, permitiendo reutilización de direcciones y operación a través de relay agents.

IPv4 header

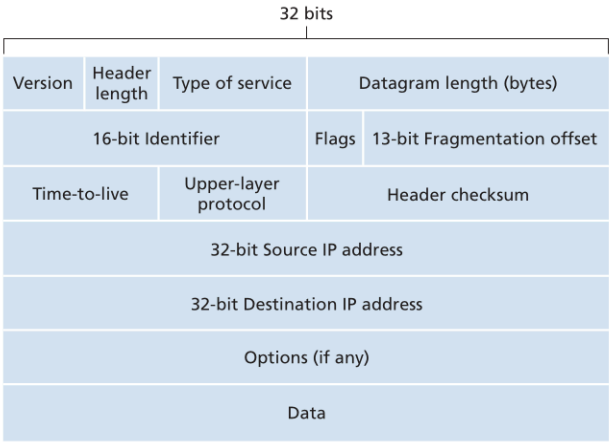


Figure 4.17 ♦ IPv4 datagram format

Versión: 4 ⇒ IPv4
Header Length: tamaño del encabezado, en palabras de 32 bits. Sirve para saber donde empiezan los datos.
Type of Service: Permite dar prioridad a ciertos paquetes.
Datagram Length (bytes): El tamaño total del paquete. Sirve para saber donde termina el paquete.
16-bit Identifier: ID del paquete, para rearmar si hubo fragmentación.
Flags: Controlan la fragmentación, DF (Don't Fragment), MF (More Fragments)
Fragment Offset (13 bits): Indica la posición dentro del paquete original.
Time To Live: Saltos restantes. Evita ciclos.
Upper-Layer protocol: protocolo de la capa de arriba. TCP, UDP o ICMP.
Header Checksum: Verifica errores del header, no de los datos. Si falla, se descarta el paquete.

Descríbeme el header IPv4: El encabezado IPv4 incluye la versión, longitud, tipo de servicio, tamaño total, campos de fragmentación, TTL, protocolo superior, checksum, direcciones origen y destino, opciones y los datos.

IPv6 diferencias

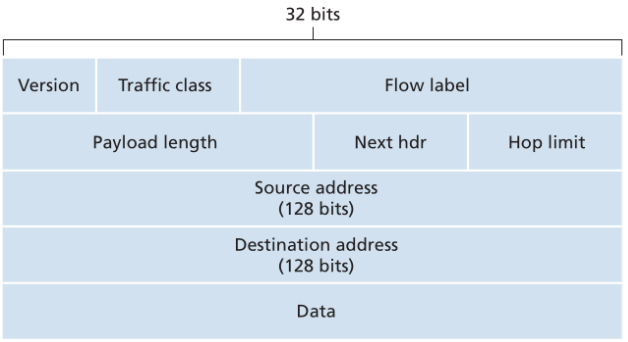


Figure 4.26 ♦ IPv6 datagram format

Versión: 6 ⇒ IPv6.
Traffic class: Priorizar tráfico.
Flow label: Indica el flujo de paquetes, es para que los routers traten igual a todos los paquetes de una conexión, ej: streaming.
Payload length: Tamaño de los datos, sin el header.
Next header: Indica que protocolo viene después, como TCP, UDP o ICMP.
Hop limit: TTL en IPv4.
Las IPs de origen y destino son más grandes que en IPv4.

Explicame el header de IPv6: El encabezado IPv6 es fijo de 40 bytes e incluye versión, traffic class, flow label, longitud del payload, next header, hop limit y direcciones origen y destino de 128 bits. No incluye checksum ni fragmentación en routers, lo que simplifica el reenvío.
Motivación principal: Falta de direcciones (Al ser las direcciones de 128 bits en vez de 32, hay más direcciones), Fragmentación prohibida en los routers, sólo los hosts pueden hacerlo (mejorando la retransmisión en routers), Auto-configuración, sin checksum.
Si me preguntan, ¿por qué IPv6? ⇒ Porque el espacio de direcciones en IPv4 con 32 bits resultó insuficiente ante el crecimiento de internet. IPv6 amplía el espacio a 128 bits, eliminando la necesidad de NAT masivo y permitiendo escalabilidad global.

IPv4 vs IPv6

IPv4 y IPv6 se diferencian principalmente en el tamaño del espacio de direcciones.
IPv4 utiliza direcciones de 32 bits, lo que permite aproximadamente cuatro mil millones de direcciones, las cuales resultaron insuficientes debido al crecimiento de Internet.
En cambio, IPv6 utiliza direcciones de 128 bits, proporcionando un espacio prácticamente ilimitado.

Debido a esta escasez, en IPv4 es necesario el uso masivo de NAT para compartir direcciones públicas, lo cual rompe el principio end-to-end.
En IPv6, gracias a la abundancia de direcciones, no es necesario utilizar NAT y se recupera la comunicación directa entre hosts.

En cuanto al encabezado, IPv4 posee un header de tamaño variable, entre 20 y 60 bytes, que incluye campos como checksum, fragmentación y opciones.
IPv6, en cambio, tiene un encabezado fijo de 40 bytes, más simple, sin checksum ni opciones internas, lo que acelera el procesamiento en los routers.

Respecto a la fragmentación, en IPv4 puede ser realizada tanto por los hosts como por los routers intermedios.
En IPv6, la fragmentación sólo puede ser realizada por el host emisor, nunca por los routers, mediante extensión headers.

En IPv4 existe el campo TTL, que limita la cantidad de saltos de un paquete, mientras que en IPv6 este campo se reemplaza por Hop Limit, con la misma función.

Para identificar el protocolo de capa superior, IPv4 utiliza el campo Protocol, mientras que IPv6 utiliza Next Header, que además permite encadenar múltiples encabezados de extensión.

En relación con la configuración, en IPv4 suele utilizarse DHCP o configuración manual.
En IPv6 existe además la autoconfiguración sin estado, llamada SLAAC, que permite a los hosts configurarse automáticamente.

En términos de calidad de servicio, IPv6 mejora el soporte mediante los campos Traffic Class y Flow Label, permitiendo un mejor tratamiento de flujos.

En cuanto a seguridad, en IPv4 el uso de IPsec es opcional, mientras que en IPv6 fue diseñado con soporte integrado para este mecanismo.

En síntesis, IPv6 surge principalmente para resolver la escasez de direcciones de IPv4, simplificando el encabezado, eliminando la fragmentación en routers y reduciendo la necesidad de NAT, lo que permite una red más escalable y eficiente.

Versión corta: IPv6 surge principalmente para resolver la escasez de direcciones de IPv4, ampliando el espacio a 128 bits y simplificando el encabezado, eliminando checksum, fragmentación en routers y reduciendo la necesidad de NAT.

SDN (Software Defined Networks): Separa el plano de control del de datos, permitiendo programar la red de forma centralizada.

Middle Boxes: NAT (Network Address Translation), Firewalls
NAT: Es un mecanismo que traduce direcciones y puertos entre redes privadas y públicas, permitiendo que múltiples hosts comparten una única IP pública. Funciona mediante una tabla de traducción y rompe el principio end-to-end, aunque permite extender la vida útil de IPv4.
NAT es en router de borde (gateway), está entre Red privada e Internet.
Un ejemplo puede ser un hogar:
Yo en mi hogar tengo la IP 123.45 y quiero preguntarle algo a google que tiene ip 876.54.
Hago esta petición, va al router y mi router tiene una IP pública: 200.10
Mi router cambia la petición de:
Src: 123.45

Dst: 876.54
a:
Src: 200.10
Dst: 876.54
y guarda en su tabla:
Privada: 123.45 ⇒ Pública: 876.54
Google responde:
Dst: 200.10
NAT mira la tabla y la vuelve:
Dst: 123.45

Se dice que rompe P2P porque internet fue pensado Host a Host y ahora con NAT, pasa que es Host a NAT a Host, agregando complejidad.
También se dice que NAT es seguridad porque oculta IPs, pero no es seguridad porque no filtra tráfico, la seguridad real es un firewall. IPv6 elimina NAT.

Firewall: Un Firewall es un sistema que filtra tráfico de red según reglas de seguridad. Puede ser
- stateless: Solo mira IP origen y destino, puerto, protocolo. Por ejemplo, una regla sería Permitir: 200.10.1.5 ⇒ puerto 80. Es rápido y simple pero no sabe si es parte de una conexión.
- stateful: Además de mirar paquetes, guarda el estado de las conexiones. Si vos abrís una conexión ⇒ deja volver la respuesta. Si alguien entra sin permiso ⇒ bloquea.
Es más seguro e inteligente pero usa más memoria y es más complejo.
- de aplicación: Analiza el contenido, por ejemplo, HTTP, FTP, DNS. Puede bloquear URLs, archivos, comandos. Ejemplo, Proxy.

Se suele utilizar stateful que mantienen el estado de las conexiones y permiten mayor seguridad. A diferencia de NAT, su función principal es el control de acceso.

IANA: es el organismo encargado de administrar globalmente las direcciones IP, la zona raíz del DNS y los números de protocolos, garantizando la unicidad y coordinación de Internet.
Internet está dividido en 5 zonas, quienes reciben los números de IANA y los distribuyen.

Capitulo 5, The Network Layer: Control Plane

Plano de control: El plano de control es la parte de la red que se encarga de decidir por dónde van a circular los paquetes. El plano de control:
- Calcula rutas
- Mantiene tablas
- Ejecuta protocolos de ruteo
- Decide el camino

NO mueve paquetes

Hay distintos tipos:
Distribuido vs Centralizado

Distribuido (Internet clásico): Cada router habla con vecinos, intercambia información, calcula su tabla. Por ejemplo, OSPF o BGP. Cada router toma decisiones locales con información parcial.

Centralizado (SDN): Hay una entidad **controlador** que: Ve toda la red, Decide rutas y Programa switches. Por ejemplo, SDN. En SDN las decisiones se concentran en un controlador.

Estático vs Dinámico

Estático: Rutas fijas, Configuradas a mano, No cambian. Por ejemplo, Default gateway.

Dinámico: Se adapta, Recalcula, Reacciona a fallas. Por ejemplo, BGP, OSPF. Internet funciona casi exclusivamente con ruteo dinámico

Organizacion de Internet

AS: Sistemas Autónomos, es un conjunto de redes bajo una misma administración. Por ejemplo, Telecom, Google, UBA. Cada AS tiene un número ASN. Este número ASN sirve para identificar de manera única a un Sistema Autónomo dentro de Internet y permitir que los protocolos como BGP (sist. distri) intercambien información de ruteo entre AS.

ISP y Jerarquía: Internet no es plano, es jerárquico. Existe Tier 1(ISP-global), Tier 2(Regionales), Tier 3(Locales), Usuarios.

Relaciones: Los Sistemas Autónomos se relacionan mediante acuerdos comerciales. En una relación de **transit**, un AS le paga a otro para obtener conectividad global, y el proveedor reenvia tráfico hacia cualquier destino.

En cambio, en el **peering** dos AS intercambian tráfico entre sus propias redes sin pagar, pero no envían tráfico de terceros.

Estas relaciones se reflejan directamente en BGP, que prioriza políticas comerciales por sobre métricas técnicas, es decir, se seleccionan rutas según políticas económicas por sobre distancia.

Protocolos de ruteo
Los **protocolos** de **enrutamiento** lo único que hacen es encontrar el camino al destino, sin tener en cuenta si tiene la menor pérdida, distancia, etc. Permiten que los routers construyan automáticamente sus tablas de forwarding intercambiando información entre ellos.
Distintos puntos de vista: link-state, distance-vector
Distance-Vector: Cada router solo conoce el costo hacia sus vecinos, les pregunta cuánto les cuesta llegar a cada destino, actualiza su tabla en base a eso. Usa el algoritmo de Bellman-Ford. Cuando se cae un enlace, los routers tardan en enterarse y se generan loops infinitos.

- **RIP (Routing Information Protocol)**: Utiliza distance vector, establece una métrica de número de saltos y setea un máximo de 15 saltos, si llega a 16 ⇒ infinito. Es simple pero no escala y converge lento.

Link-State: Cada router descubre quienes son sus vecinos, mide el costo hacia ellos y difunde esa información a TODOS los routers. Entonces todos tienen el mismo mapa de la red. Se hace flooding. Converge más rápido y es más escalable pero es más complejo y consume más memoria.
Cada router construye el grafo completo, corre el algoritmo de **Dijkstra** y calcula el camino mínimo. En link-state cada router tiene una visión global de la topología. Ej: OSPF (Open Shortest Path First) puede trabajar con pesos (Capacidades o prioridades) en los enlaces, autenticación, zonas, multicast.

SDN (Software De ned Networks), Plano de Control

Se separan las decisiones de ruteo del hardware y se llevan a un controlador central. Ningún router decide por sí solo.

Separación del plano de datos y de control

Los switches no calculan rutas, solo obedecen reglas. El plano de control decide y el plano de datos reenvía.

Control de Red: externo a los conmutadores

El control se encuentra en un **controlador SDN**, estos ven toda la red, toman las decisiones y programa dispositivos.
Los switches no corren OSPF, BGP, no calculan caminos. Solo preguntan, qué hago con este paquete?

Reenvío basado en flujos

En el reenvío basado en flujos no se mira solo la IP destino (como es tradicionalmente), sino un conjunto de campos del paquete, como IP origen, IP destino, puertos, protocolo, etc., y en base a eso se decide cómo reenviar.

Red Programable

La red se controla con un software programable, es decir, SDN convierte a la red en una plataforma programable.

Cómo funciona SDN en la práctica:

Paso 1
Llega un paquete al switch.
Paso 2
El switch busca regla, si no hay ⇒ consulta al controlador.
Paso 3
El controlador decide.
Paso 4
Instala regla.
Paso 5
Los próximos paquetes ya no preguntan.

Hay varios casos, uno el switch instala la regla solo al switch que pregunto, dos instala a varios switches (todos los switches del recorrido) programando todo el camino extremo a extremo, y el último modo proactivo instala reglas sin esperar preguntas.

OpenFlow es el protocolo que usa SDN para hablar con los switches, es el lenguaje.

Explicame SDN:
SDN se propone separar el plano de control del plano de datos. Las decisiones de ruteo se trasladan a un controlador externo, mientras que los switches solo ejecutan reglas.

El reenvio se basa en flujos, que consideran múltiples campos de un paquete y no solo la IP destino. Esto permite que la red sea programable, facilitando la automatización y el control centralizado.
Aunque presenta desventajas como escalabilidad, punto único de falla y latencia inicial.

BGP: Border Gateway Protocol, es el responsable del enrutamiento a nivel global, EL protocolo del plano de control global. Une los AS entre sí, ojo no se usa dentro de AS (aca es OSPF/RIP), sino, entre AS ⇒ Internet global.

BGP no intercambia costos, sino que intercambia **Prefijos IP** (Qué redes existen), **AS_PATH** (Por que AS se pasa), **Atributos** (Preferencias, políticas).
BGP anuncia prefijo junto con el camino de AS que debe recorrerse.
Ejemplo:
Red: 200.10.0.0/16
Camino: AS1 → AS5 → AS9

BGP no se usa directamente para reenviar paquetes, sino para seleccionar la mejor ruta. A partir de esa decisión, se actualiza la tabla de forwarding con un next hop.

Las tablas de forwarding solo contienen prefijo y next hop. **BGP** mantiene una tabla aparte con información adicional, como el **AS-PATH** y atributos de política. Esta información se usa para seleccionar la mejor ruta, que luego se instala en la tabla de forwarding.

Internet está compuesta por Sistemas Autónomos (Red de redes) que se interconectan entre sí. Cuando queremos enviar tráfico desde un host en un AS hacia otro host en un AS diferente, el paquete debe atravesar una secuencia de AS intermedios.

Los routers de borde ejecutan **BGP**, que es el protocolo encargado de intercambiar información de rutas entre AS.

Supongamos que el AS3 es dueño de la red 200.10.0.0/16. AS3 anuncia a sus vecinos BGP que puede llegar a esa red, enviando el prefijo junto con el AS-PATH inicial que contiene sólo su propio ASN.

Si AS2 recibe ese anuncio, agrega su ASN al AS-PATH y lo reanuncia a sus propios vecinos. Finalmente, AS1 puede recibir un anuncio indicando que la red 200.10.0.0/16 es alcanzable a través del camino AS2 → AS3.

Con esa información, AS1 puede decidir si esa es la mejor ruta según sus políticas.

¿Cuándo se envían estos anuncios? ⇒ **BGP** no transmite anuncios de manera periódica, sino que funciona por eventos. Un AS anuncia sus prefijos cuando se establece una sesión **BGP** o cuando hay cambios en su topología. Las redes internas suelen agregarse en prefijos más grandes para reducir la cantidad de anuncios (no se anuncian redes pequeñas). Además, si una ruta deja de estar disponible, se envía un mensaje de retiro.

¿Qué es el Path pre pending? ⇒ Consiste en repetir artificialmente el ASN en el AS-PATH para hacer una ruta menos atractiva y así influir en la selección de rutas. Se utiliza para balancear tráfico, evitar congestión, controlar por donde entran a tu red.

BGP también sirve para detectar bucles.
BGP utiliza TCP para garantizar la fiabilidad.

Existent Reglas **Mutually Agreed Norms for Routing Security**

1. **Filtering:** Se establecen reglas de confiabilidad entre la información que se brinda sobre los anuncios entre AS's. El operador de red es capaz de comunicar que los anuncios son correctos.
2. **Anti-Spoofing:** El operador de red implementa un protocolo para detectar direcciones IP de origen falsas.
3. **Coordination:** El operador de red mantiene información de contacto actualizada y accesible globalmente.
4. **Global Validation:** Un operador publica documentación de su política de ruteo, ASNs y prefijos, que se anunciaran a terceros, facilita la información.

¿Qué pasa con BGP si se cae un enlace?
⇒ Si el enlace físico se cae, el router BGP detecta la pérdida de conectividad, la sesión TCP utilizada por BGP se interrumpe y se eliminan todas las rutas aprendidas a través de ese vecino. Luego se recalculan las rutas utilizando alternativas si existen.

ICMP: Internet Control Message Protocol, es el protocolo que usa la red para informar errores y estados, no para enviar datos.
ICMP está asociado a **IP**, es decir, viaja dentro de paquetes **IP**. Es parte de la capa de red.
ICMP reporta errores como, no existe el destino, se cayó un router, TTL agotado, puerto cerrado.
ICMP es utilizado en ping, traceroute.
ICMP brinda información de estado.
ICMP informa errores, pero no los soluciona.

Muchos firewalls bloquean ICMP porque se puede usar para reconocimiento, ataques. Pero bloquear ICMP completamente puede afectar el funcionamiento de la red.

ICMP NO TIENE PUERTOS!!

Resumen: **ICMP** es un protocolo de control asociado a IP que se utiliza para reportar errores y estados de red. No transporta datos de aplicación ni utiliza puertos. Se emplea en herramientas como ping y traceroute mediante mensajes Echo y Time Exceeded. ICMP informa problemas, pero no los corrige.

Puedo averiguar las IP's de los router intermedios entre un host y otro, a su vez, el tiempo entre cada uno.

Capitulo 6, The Link Layer and LANs // Capitulo 7, Wireless and Mobile NetWorks

La capa de enlace se encarga de la comunicación entre dispositivos/nodos directamente conectados. Encapsula los datos en tramas, controla el acceso al medio compartido, detecta errores y utiliza direcciones MAC. Funciona salto a salto y se reencapsula en cada enlace.

Servicios
Framing (Encapsulado): Toma el paquete IP y lo mete en una trama. Estructura general: Header | Datos | Trailer
El trailer suele tener el CRC.
Entrega confiable (Opcional): Detectan errores, envían automáticamente. Ej: WIFI. Otros no tienen esto, ej: Ethernet moderno.
Control de flujo (Opcional): Evitar que el receptor se sature.
Detección de errores: Detecta si hubo corrupción en el enlace físico. No corrige necesariamente, solo detecta.

Tipos de enlaces
Enlace punto a punto: Solo dos dispositivos, Ej: Router ↔ Router. No hay competencia por el medio.
Enlace de acceso múltiple: Muchos dispositivos comparten el medio, por ej: Wifi, Ethernet con hub. Acá aparece el problema MAC.
En enlaces compartidos es necesario un protocolo de acceso al medio.

Framing: ¿Por qué no mandar bits sueltos? Porque necesitamos:
- Saber dónde empieza y termina
- Identificar destinatario
- Detectar errores

Una trama Ethernet típica tiene:
- MAC destino
- MAC origen
- Tipo
- Datos
- CRC

Detección de errores
Se utiliza:
- Paridad
- Checksum
- CRC

Lo importante es **CRC** (Cyclic Redundancy Check) y la idea es:
- Se calcula un valor matemático

- Se manda junto con los datos
- El receptor recalcula
- Si no coincide ⇒ Error.

BER: Bit Error Rate, es la tasa de error de bits, es decir, la probabilidad de que un bit transmitido llegue incorrectamente. Se calcula como la cantidad de bits erróneos sobre el total transmitido y refleja la calidad del enlace físico.
A menor **BER** ⇒ mejor el enlace.

MAC: Cuando el medio es compartido puede haber colisiones, necesitamos reglas. Acá aparecen **CSMA/CD** y **CSMA/CA**

Protocolos de Acceso al Medio (MAC)
En muchos enlaces como: Wifi, Ethernet viejo (con hub) y redes compartidas, muchos dispositivos usan el mismo medio físico. Si todos transmiten a la vez ⇒ colisión. Entonces la capa de enlace necesita reglas para "Quien habla y cuando" ⇒ eso es **MAC**.
"Los protocolos MAC regulan el acceso a un medio compartido."

CSMA/CD
Se usaba en Ethernet con HUB (Medio compartido). Hoy casi no se usa porque con switches no hay colisiones.
Que significa CSMA/CD
- Carrier ⇒ medio
- Sense ⇒ escuchar
- Múltiple ⇒ varios
- Access ⇒ acceso
- Collision ⇒ colisión
- Detection ⇒ detección

"Escucho, transmito y detecto colisiones"
Algoritmo
Paso 1:
Escucha al medio. ¿Está libre?
- No ⇒ espero
- Si ⇒ sigo

Paso 2:
Empieza a transmitir.
Paso 3:
Mientras transmite, escucha. ¿Hay colisión?
- No ⇒ sigo.
- Si ⇒ **Paso 4**

Paso 4 (si hay colisión)
- Manda senal de jam
- Corta transmisión
- Espera tiempo aleatorio
- Reintenta

Eso es Backoff exponencial

El **Problema** de **CSMA/CD** es que solo funciona si puede escuchar mientras transmite, en WIFI no se puede, por eso ahí no sirve. **CSMA/CD** detecta colisiones mientras transmite.

CSMA/CA
Se usa en **Wifi (802.11)**
Que significa CSMA/CA

- Collision
- Avoidance

"Trato de evitar colisión antes."
No la detecta. La previene.
¿Por qué no se puede detectar? ⇒ Porque en Wifi estás transmitiendo, no podes escuchar bien al mismo tiempo. Entonces no sabes si chocaste.
Algoritmo
Paso 1:
Escucha el canal. Libre?

- No ⇒ espero
- Si ⇒ sigo

Paso 2:
Espero tiempo aleatorio (Backoff)
Aunque esté libre.
Para evitar que dos manden juntos.
Paso 3 (opcional): RTS/CTS
Para evitar "terminal oculto":

1. **RTS: "**Puedo mandar?"
2. **CTS:** " Si, manda"
3. Transmite.

Paso 4:
Envía datos.
Paso 5:
Espera ACK

- Llega ⇒ OK
- No llega ⇒ reintenta

"**CSMA/CA** evita colisiones usando backoff y confirmaciones."

"Explique CSMA/CD y CSMA/CA" ⇒ En enlaces compartidos se utilizan protocolos MAC. En Ethernet clásico se usa CSMA/CD, donde los dispositivos son capaces de escuchar en el medio, transmiten y detectan colisiones, reintentando con backoff. En redes inalámbricas se utiliza CSMA/CA porque no se puede escuchar a la vez que transmitis, entonces en lugar de detectar colisiones, las evita. Las evita mediante tiempos aleatorios, RTS/CTS y confirmaciones."

Explicación de CSMA/CA sin RTS/CTS (como quiere el profe): RTS/CTS es un mecanismo de más que ayuda a evitar colisiones en infraestructuras. La esencia del algoritmo es la siguiente:
Supongamos dos **estaciones A** y **B** que ambas utilizan el **Access Point AP.**
A se conecta al enlace compartido y si **no** detecta señal ⇒ entiende que está **libre**.

Espera un tiempo "**Difs**" fijo y luego, tiene un **contador** elegido **aleatoriamente.** Lo va descontando cada unidad de tiempo y luego empieza a mandar.
Este tráfico que va de **A** hacia **AP**, **AP** también lo reenvía a **B** (que también está conectado al enlace) para que sepa que se está usando el medio.
Una vez que **B** deja de recibir, entiende que ya no se usa.
Espera un tiempo "**Difs**" fijo y luego, también tiene un **contador** elegido **aleatoriamente.** Lo va descontando cada unidad de tiempo y luego empieza a mandar.
Pasado ese tiempo, empieza a mandar paquetes de **B** a **AP**.
AP reenvía a **A** para que sepa que se está usando el enlace compartido.

Este tiempo **Difs fijo** que se espera, es para que, si se terminó de mandar, por ejemplo, paquetes de la estación **A** hacia **AP**, esperar que **AP** mande el ACK correspondiente y no se choque con la transmisión de **B** hacia **AP**. También está el **contador aleatorio** y es porque si justo ambos quieren empezar a transmitir a **AP** (porque está libre), el que tenga el **contador menor** transmite primero y le llega señal de que está ocupado al que tenga **contador mayor**, evitando colisión.

Si hubiera **RTS/CTS,** lo que cambia es que las estaciones **A** y **B** mandan **RTS** ("Quiero transmitir durante T tiempo") a **AP** y responde **CTS** ("Ok, reserva el canal por T"). Todos escuchan esta petición y saben que está ocupado el enlace. Todo lo demás sigue igual, todos esperan un tiempo **difs**, etc, etc.

LAN: Local Area Network, es una red de área local que conecta dispositivos en un espacio reducido, como una casa u oficina, generalmente con alta velocidad, baja latencia y administración privada, usando tecnologías como Ethernet o Wifi.

```
PC ─┐
Cel ─┼─ Router ─ Internet
TV ─┘
```

Todo antes del router = LAN

Ethernet: La tecnología más usada para redes locales cableadas (LAN), funciona en capa de enlace y usa Tramas, Direcciones MAC y Switches.
Una trama tiene:
MAC destino | MAC origen | Tipo | Datos | CRC
Se reenvía por **MAC**, **no** por **IP.**

¿Qué hace un switch? ⇒ Un switch es un dispositivo de capa 2 que reenvía tramas según direcciones **MAC**. NO usa IP, NO usa BGP y NO rutea. Solo mira **MAC.**

¿Cómo aprende un switch? (Aprendizaje MAC) ⇒ Un switch tiene una tabla **Tabla MAC (CAM table)** en donde guarda **MAC** → **Puerto**
Paso 1: Llega una trama
A manda a B
El switch recibe:
Origen: MAC_A
Destino: MAC_B
Puerto: 1
Paso 2: Aprende

El switch guarda:
MAC_A → Puerto 1
Siempre aprende del **origen.**
Paso 3: Decide
Busca MAC_B

- **Caso A: La conoce**
  Reenvia solo por ese puerto.
- **Caso B: No la conoce**
  Hace flooding (envía a todos menos del que llegó).

Flooding = broadcast controlado.
Se usa cuando: No conoce destino, ARP, Broadcast.

Switch vs Hub:

Hub	Switch
Repite todo	Filtra
Colisiones	No
Lento	Rápido

Con **Switches**: cada enlace es punto a punto, entonces no hay CSMA/CD y no hay colisiones.
En Ethernet con switch no hay colisiones. Ya que cada puerto es un enlace independiente.
“Con switches, cada enlace es punto a punto y full-duplex, por lo que no existen colisiones.”

ARP: Address Resolution Protocol, soluciona el paso de la capa de Red a la capa de Enlace. Relaciona una dirección **MAC** con una de **IP**.

Problema que resuelve ARP:
PC quiere mandar a:
IP: 192.168.1.1
pero solo sabe la IP y pregunta: Que MAC tiene esa IP?
Paso 1: ARP Request (broadcast)
A manda:
¿Quién tiene 192.168.1.1?
a TODOS.
MAC destino: FF:FF:FF:FF:FF:FF
Paso 2: ARP Reply (unicast)
El **dueño** responde:
Soy yo. Mi MAC es XX:XX
Solo a **A.**
Paso 3: Cache ARP
A guarda:
IP → MAC
En su tabla **ARP**. No pregunta cada vez.

“ARP utiliza broadcast para preguntar y unicast para responder.”
El host que envía el ARP aprende la relación IP-MAC a partir de la respuesta. El switch solo aprende direcciones MAC de origen, y el router solo responde si la consulta es para él.

Las **tablas ARP** tienen tiempo de vida. Se borran.

ARP y routers
Si quiero mandar fuera de mi red, no busco **MAC** del destino final. Busco **MAC** del gateway. Ejemplo: PC → Google. ARP busca MAC del router.

Ejemplo completo:
A quiere mandar a **B**
Paso 1:
A sabe **IP_B**
Paso 2:
ARP → obtiene MAC_B
Paso 3:
A arma trama con **MAC_B**
Paso 4:
Switch aprende **MAC_A**
Paso 5:
Switch reenvía a **B**

ARP entre redes: Cuando el destino está en otra red, el host no busca la MAC del destino final, sino la del gateway (la puerta de salida, router). Primero resuelve por ARP la dirección MAC del router, le envía la trama (esta trama contiene: MAC destino = MAC del router; MAC origen = MAC del host emisor; IP destino = IP host receptor), y luego el router saca cabecera Ethernet, mira IP destino, busca ruta y sale hacia una interfaz de esa red. Luego el router realiza un nuevo ARP en la red destino para reenviar el paquete al host final.

Capítulo 8, Security in Computer Networks

Las redes necesitan seguridad porque cualquier persona conectada puede intentar interceptar, modificar o suplantar comunicaciones.

Tres pilares fundamentales
Confidencialidad: Que solo el destinatario pueda leer el mensaje. Por ejemplo: HTTPS, WhatsApp, VPN. Se logra con **cifrado**.
Integridad: Que el mensaje no sea modificado en el camino. Por ejemplo: Yo mando “Transferir $100”, no debe llegar “Transferir $1000”. Se logra con **hash + MAC / firmas digitales**.
Autenticación: Verificar quien es quien. Ejemplo: Yo se que hablo con Google y no con un falso Google. Se logra con **Certificados, Claves, TLS**.
Prevención y detección: Evitar ataques o detectarlos. Por ejemplo: **Firewalls, IDS, IPS**.

Problemas que enfrenta la seguridad
Logica (tipos de ataques): hay Sniffing (espiar), Spoofing (suplantar), DoS, MITM.

Eficiencia computacional: Cifrar cuesta CPU. Por ejemplo, HTTPS consume más que HTTP. Y acá surge el problema de Seguridad vs Velocidad.
Criptoanálisis: Intentar romper cifrados. Por ejemplo: Probar claves, ataques matemáticos. Por esto se actualizan los algoritmos.
Confianza: ¿En que confías? Ejemplo: Certificadoras, Bancos, servidores. Si fallan ⇒ se rompe todo.

Elementos para generar seguridad
Clave pública/privada (Asimétrica): Cada usuario tiene

- Pública ⇒ todos
- Privada ⇒ secreta

Ejemplo: TLS, Firmas digitales.
Uso: Cifrar, autenticar, firmar.
XOR: Operación binaria. Sirva para Cifrado simple y Stream ciphers. Base de muchos algoritmos.
Shrinking Generator: Es un generador pseudoaleatorio. Usa dos secuencias y una filtra a la otra.
Objetivo: Hacer claves impredecibles.

¿Redes seguras? Por capas:
Aplicación:

- Email seguro
- SSH

Transporte:

- SSL/TLS

Red:

- IPsec
- VPN

Enlace:

- WPA2
- WPA3
- VLAN

wifi con clave.
Middlewares:

- Firewalls
- IDS
- DPI

IPS: Intrusion Prevention System, Sistema de Prevención de Intrusiones. Detecta ataques y además los bloquea automáticamente. Se ubica en el trafico que pasa a traves de la IPs, esta en Internet ─ IPS ─ Red.
Como software en servidores. Actúa como antivirus, busca patrones conocidos, usa principalmente Firmas.
Busca comportamientos raros.

Firewall: El firewall controla el acceso a la red según reglas predefinidas. Es stateless, puede filtrar paquete por paquete y es Stateful, puede permitir repuestos si inicio la conexión interna.

IDS: Intrusion Detection System, detecta ataques pero no bloquea. Solo alerta. Puede estar en red o en host

DPI: Deep Packet Inspection, Inspección profunda de paquetes. No solo mira IP y Puerto, sino el contenido. Se encuentra principalmente en Capa de aplicación.

La analogía con el aeropuerto es entre **firewall**, **IPS** e **IDS**:
Una vez que entras al aeropuerto, tu equipaje tiene que pasar por las cintas en donde se escanea el equipaje, el agente puede ver si hay algo peligroso o no, pero solo viendo eso no puede hacer nada, este viene a ser el **IDS**. Y el conjunto de esta máquina que escanea y la policía que te revisa el equipaje, luego de detectar algo en la máquina, hace referencia a **IPS**, que además de detectar un “cuchillo” lo puede frenar.
Antes de subir al avión, tu carry on tiene que cumplir unas medidas, esta etapa hace referencia al **firewall**, en donde, se tienen que cumplir una regla o se filtra el tráfico.

Una analogía clara entre **firewall**, **IDS**, **IPS** y **DPI** es con un aeropuerto.

En una red, el **firewall** funciona como el control de medidas del carry on antes de subir al avión: si no cumple las reglas, no pasa. Filtra según políticas.

Luego, el equipaje pasa por una máquina que lo escanea. Esa máquina sería como un **IDS**, que detecta si hay algo sospechoso, pero solo alerta, no podes hacer nada con solo ver ese escaneo.

Si además de detectarlo interviene la policía y te detiene, eso sería como un **IPS**, que no solo detecta sino que también bloquea el ataque.

Además, el **DPI** sería como revisar manualmente el contenido de la valija, no solo su forma, permitiendo analizar en profundidad qué se está transportando.